
EEM 561 Machine Vision

Lecture 11

Recognition

Spring 2025

Asst. Prof. Dr. Onur Kılınç

Eskisehir Tech. Uni., Dept. EEE



Source:
[Charley Harper](#)

Overview

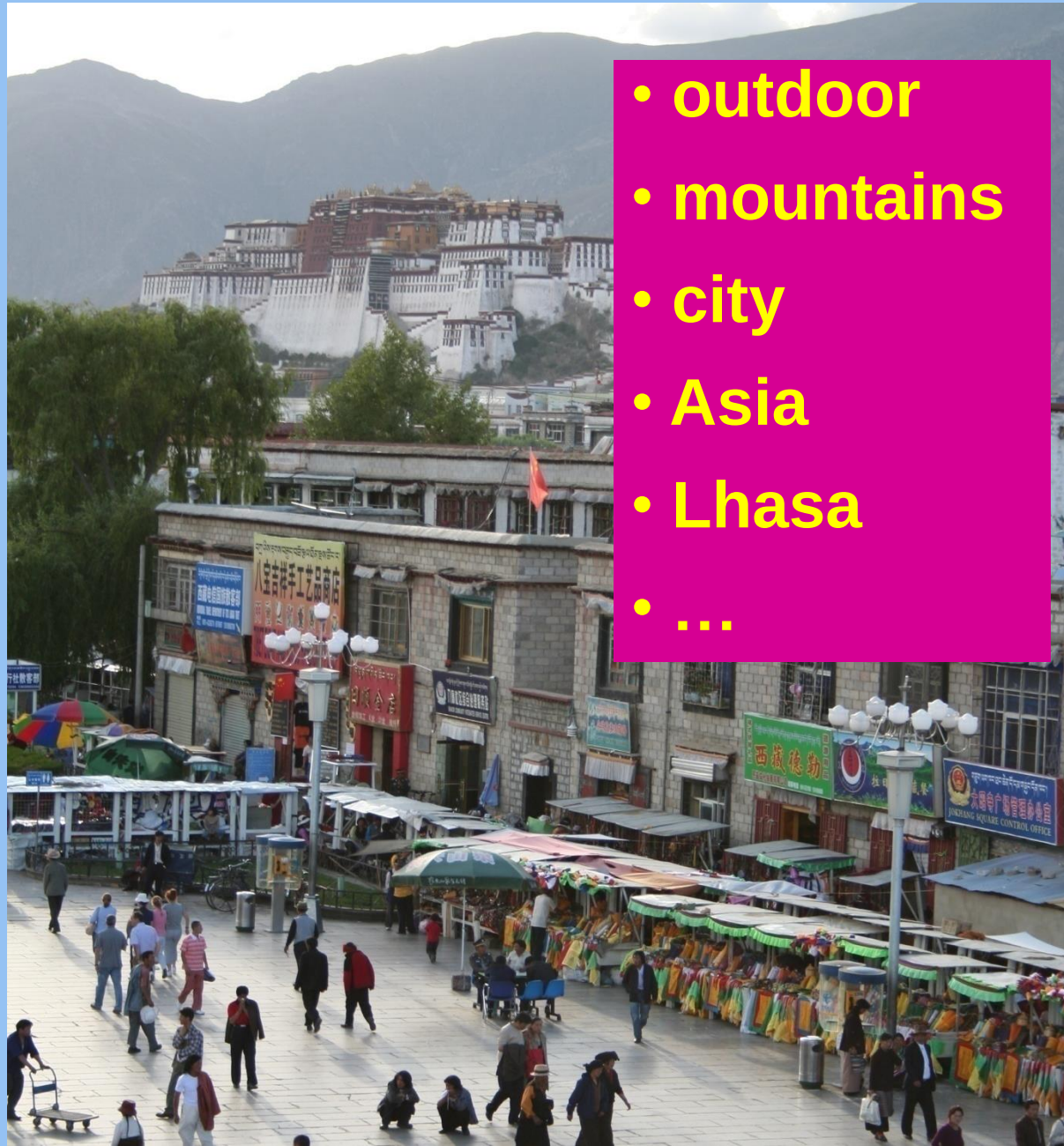
- Recognition tasks
- A statistical learning approach
- “Classic” recognition pipeline
 - Bags of features
 - Classifiers
 - Generalization
 - Best practices
- Next week: neural networks and “deep” recognition pipeline

Common recognition tasks



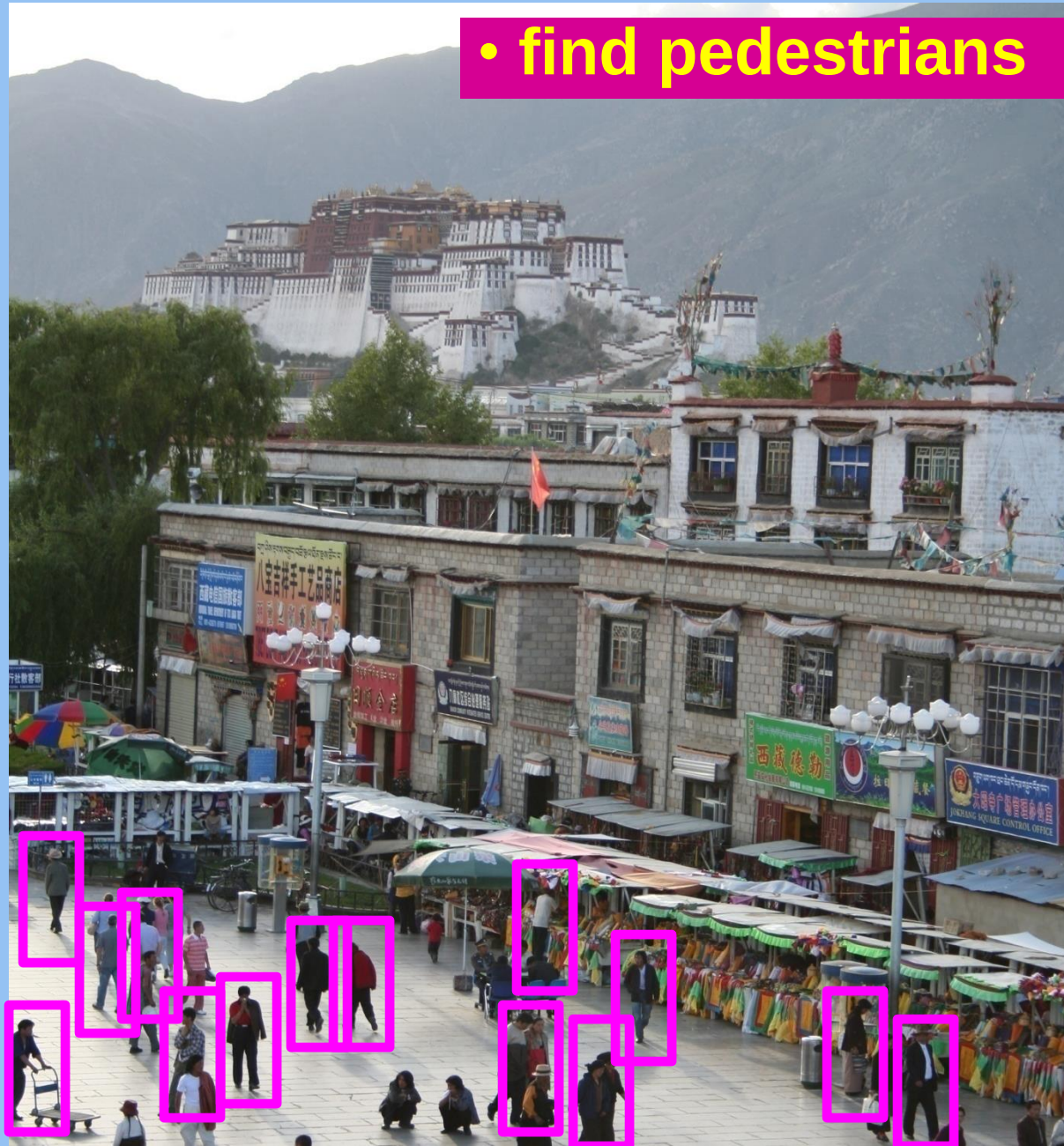
Adapted from
Fei-Fei Li

Image classification and tagging



- outdoor
- mountains
- city
- Asia
- Lhasa
- ...

Object detection



Activity recognition



Semantic segmentation

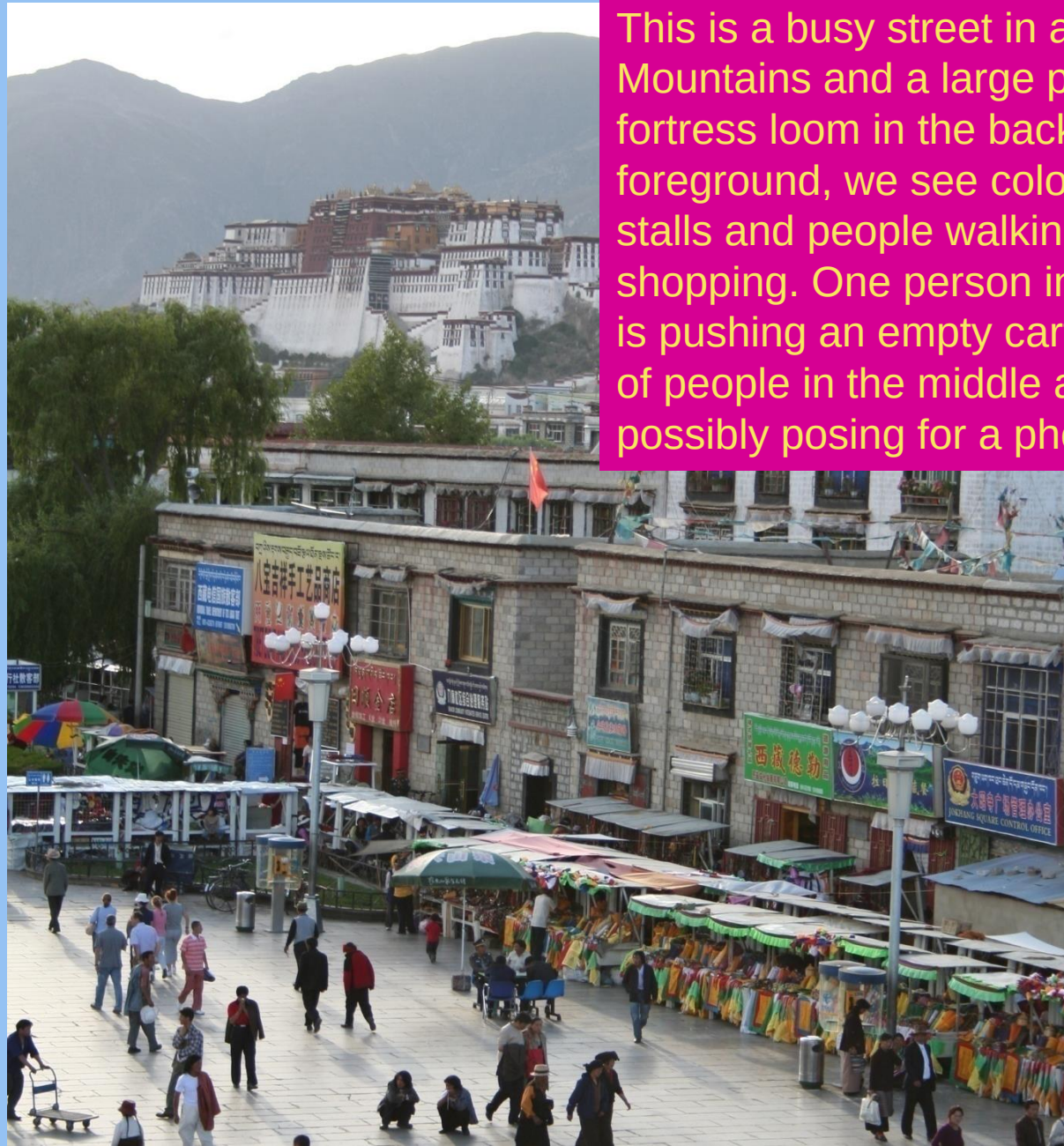


Adapted from
Fei-Fei Li

Semantic segmentation



Image description



This is a busy street in an Asian city. Mountains and a large palace or fortress loom in the background. In the foreground, we see colorful souvenir stalls and people walking around and shopping. One person in the lower left is pushing an empty cart, and a couple of people in the middle are sitting, possibly posing for a photograph.

Image classification



The statistical learning framework

- Apply a prediction function to a feature representation of the image to get the desired output:

$$f(\text{apple image}) = \text{"apple"}$$

$$f(\text{tomato image}) = \text{"tomato"}$$

$$f(\text{cow image}) = \text{"cow"}$$

The statistical learning framework

$$y = f(\mathbf{x})$$

output prediction function feature representation

- **Training:** given a *training* set of labeled examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, estimate the prediction function f by minimizing the prediction error on the training set
- **Testing:** apply f to a never before seen *test example* $\mathbf{x}$ and output the predicted value $y = f(\mathbf{x})$

Steps

Training

Training
Images

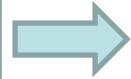
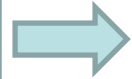


Image
Features



Training
Labels



Training



Learned
model

Testing



Test Image



Image
Features

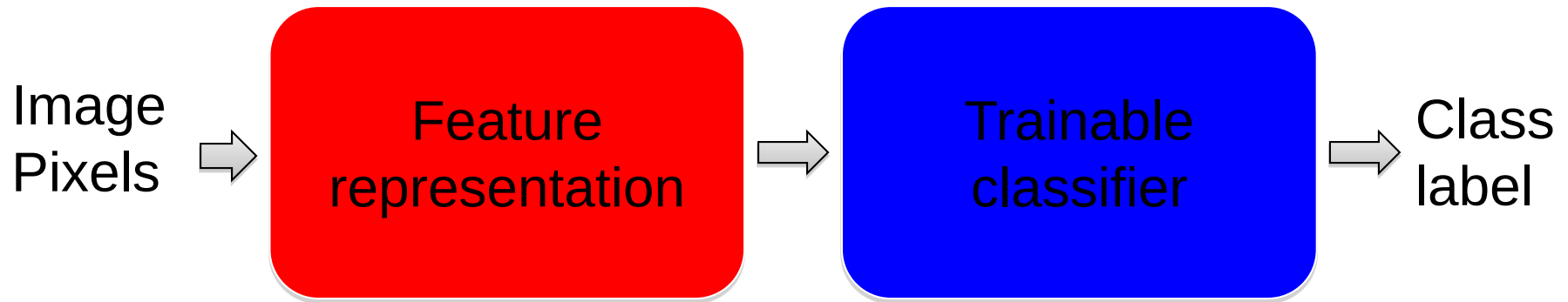


Prediction

Learned
model

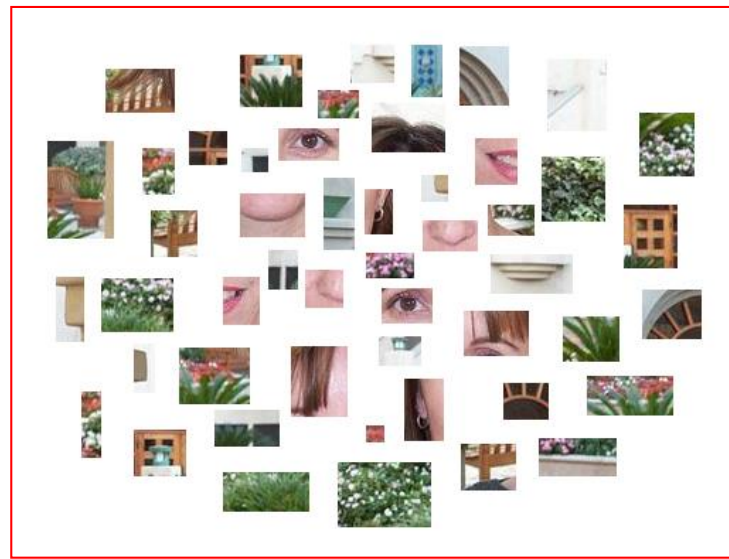
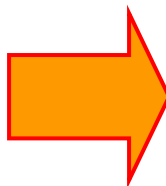


“Classic” recognition pipeline

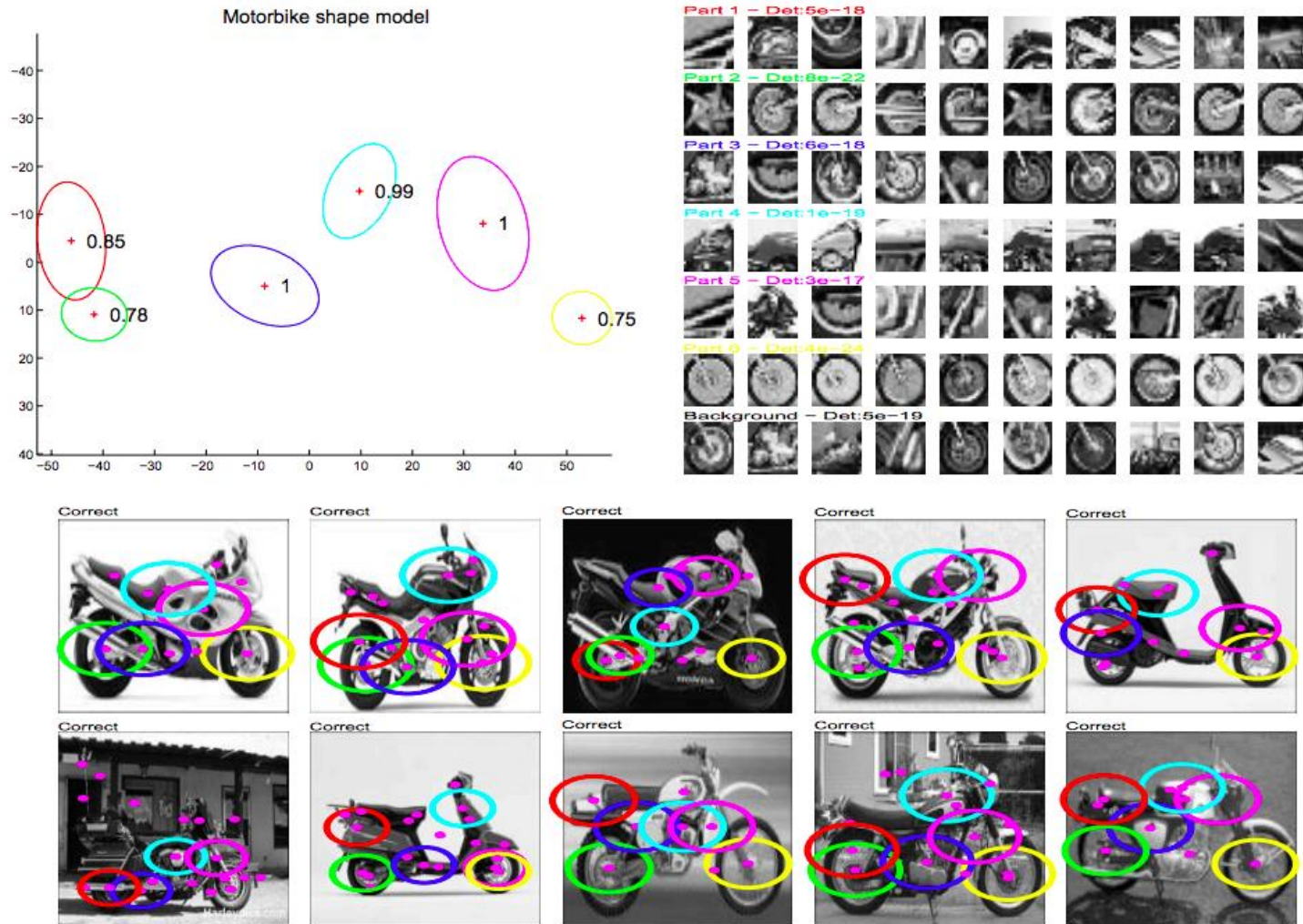


- Hand-crafted feature representation
- Off-the-shelf trainable classifier

“Classic” representation: Bag of features

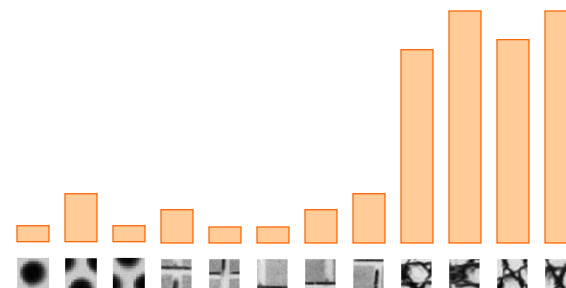
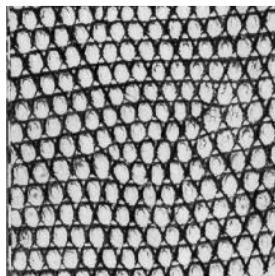
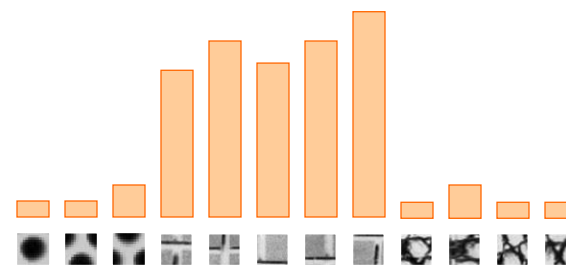
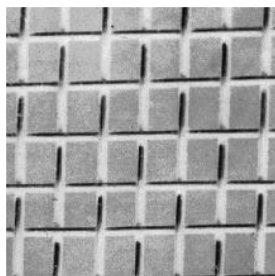
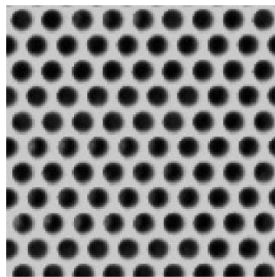


Motivation 1: Part-based models



Weber, Welling & Perona (2000), Fergus, Perona & Zisserman (2003)

Motivation 2: Texture models



Julesz, 1981; Cula & Dana, 2001; Leung & Malik 2001; Mori, Belongie & Malik, 2001; Schmid 2001; Varma & Zisserman, 2002, 2003; Lazebnik, Schmid & Ponce, 2003

Motivation 3: Bags of words

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

Motivation 3: Bags of words

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

2007-01-23: State of the Union Address

George W. Bush (2001-)

abandon accountable affordable afghanistan africa aided ally anbar armed army baghdad bless challenges chamber chaos
choices civilians coalition commanders commitment confident confront congressman constitution corps debates deduction
deficit deliver democratic deploy dikembe diplomacy disruptions earmarks economy einstein elections eliminates
expand extremists failing faithful families freedom fuel funding god haven ideology immigration impose
insurgents iran **iraq** islam julie lebanon love madam marine math medicare moderation neighborhoods nuclear offensive
palestinian payroll province pursuing **qaeda** radical regimes resolve retreat rieman sacrifices science sectarian senate
september shia stays strength students succeed sunni tax territories **terrorists** threats uphold victory
violence violent **war** washington weapons wesley

Motivation 3: Bags of words

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

2007-01-23: State of the Union Address

George W. Bush (2001-)

abandon

choices c

deficit c

expand

insurgen

palestini

septemb

violenc

1962-10-22: Soviet Missiles in Cuba

John F. Kennedy (1961-63)

abandon achieving adversaries aggression agricultural appropriate armaments **arms** assessments atlantic ballistic berlin

buildup burdens cargo college commitment communist constitution consumers cooperation crisis **cuba** dangers

declined **defensive** deficit depended disarmament divisions domination doubled **economic** education

elimination emergence endangered equals **europe** expand exports fact false family forum **freedom** fulfill gromyko

halt hazards **hemisphere** hospitals ideals **independent** industries inflation labor latin limiting minister **missiles**

modernization neglect **nuclear** oas obligation observer **offensive** peril pledged predicted purchasing quarantine quote

recession rejection republics retaliatory safeguard sites solution **soviet** space spur stability standby **strength**

surveillance **tax** territory treaty undertakings unemployment **war** warhead **weapons** welfare western widen withdraw

Motivation 3: Bags of words

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

2007-01-23: State of the Union Address

George W. Bush (2001-)

abandon

choices

deficit

expand

insurgen

palestini

septemb

violenc

1962-10-22: Soviet Missiles in Cuba

John F. Kennedy (1961-63)

abandon

build

declin

elimina

halt ha

modern

recessi

surveill

1941-12-08: Request for a Declaration of War

Franklin D. Roosevelt (1933-45)

abandoning acknowledge aggression aggressors airplanes armaments **armed** army assault assembly authorizations bombing

britain british cheerfully claiming constitution curtail december defeats defending delays democratic dictators disclose

economic empire endanger **facts** false forgotten fortunes france **freedom** fulfilled fullness fundamental gangsters

german germany god guam harbor hawaii hemisphere hint hitler hostilities immune improving indies innumerable

invasion **japanese**

islands isolate labor metals midst midway navy nazis obligation offensive

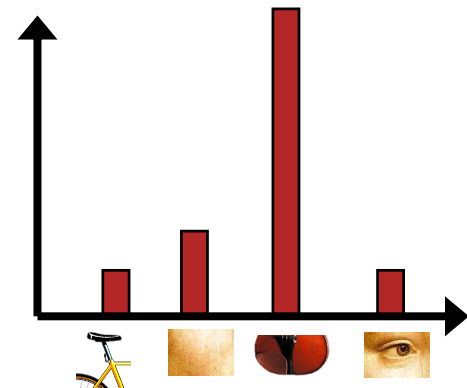
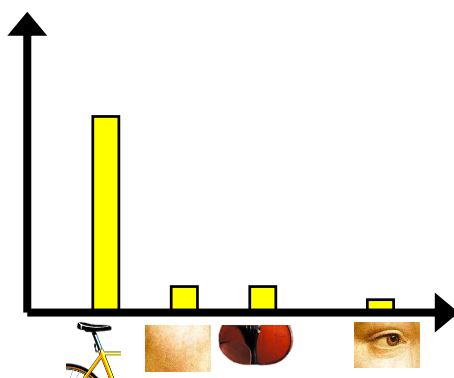
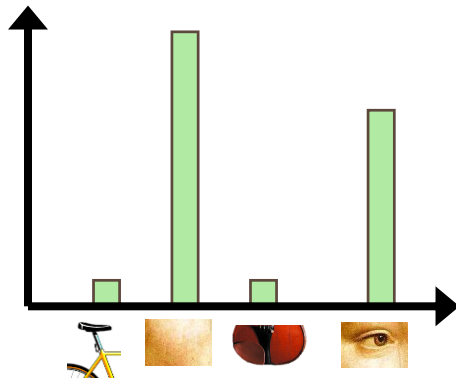
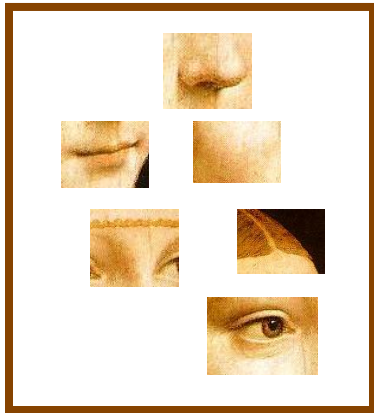
officially **pacific** partisanship patriotism pearl peril perpetrated perpetual philippine preservation privilege reject

repaired **resisting** retain revealing rumors seas soldiers speaks speedy stamina **strength** sunday sunk supremacy tanks taxes

treachery true tyranny undertaken victory **war** wartime washington

Bag of features: Outline

1. Extract local features
2. Learn “visual vocabulary”
3. Quantize local features using visual vocabulary
4. Represent images by frequencies of “visual words”

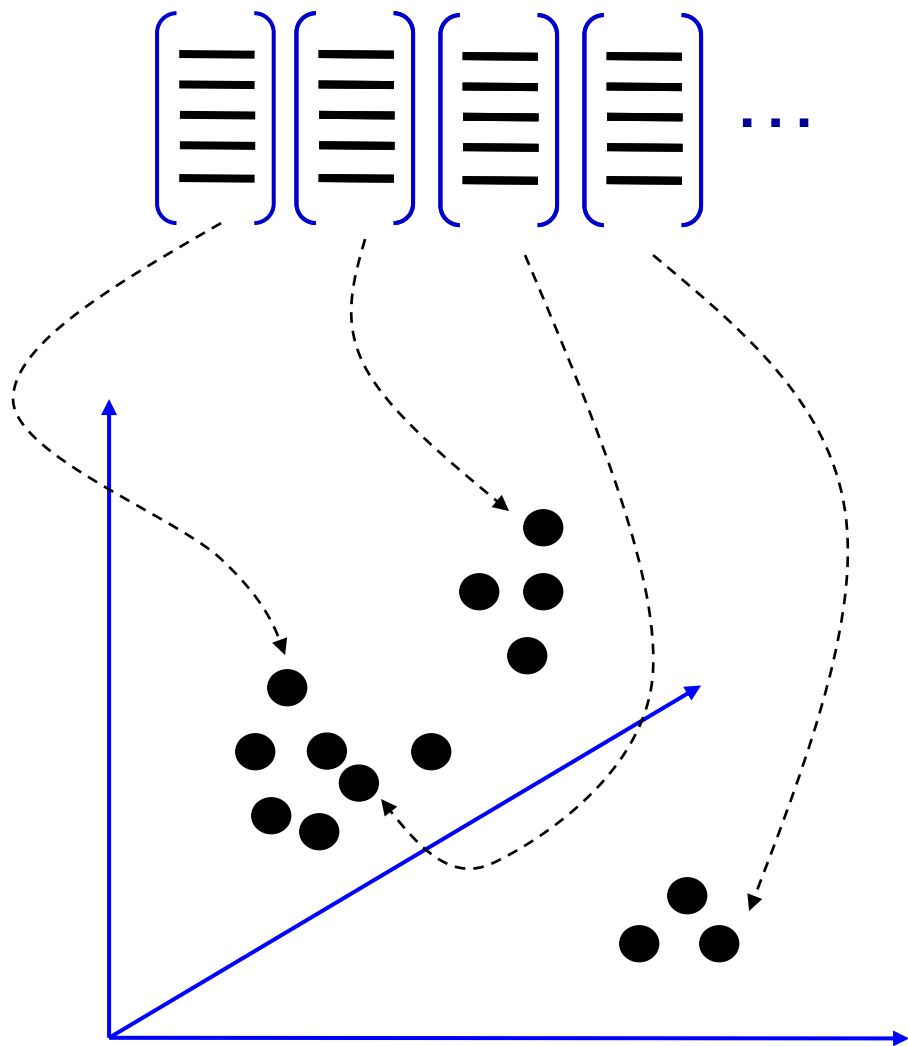


1. Local feature extraction

- Sample patches and extract descriptors

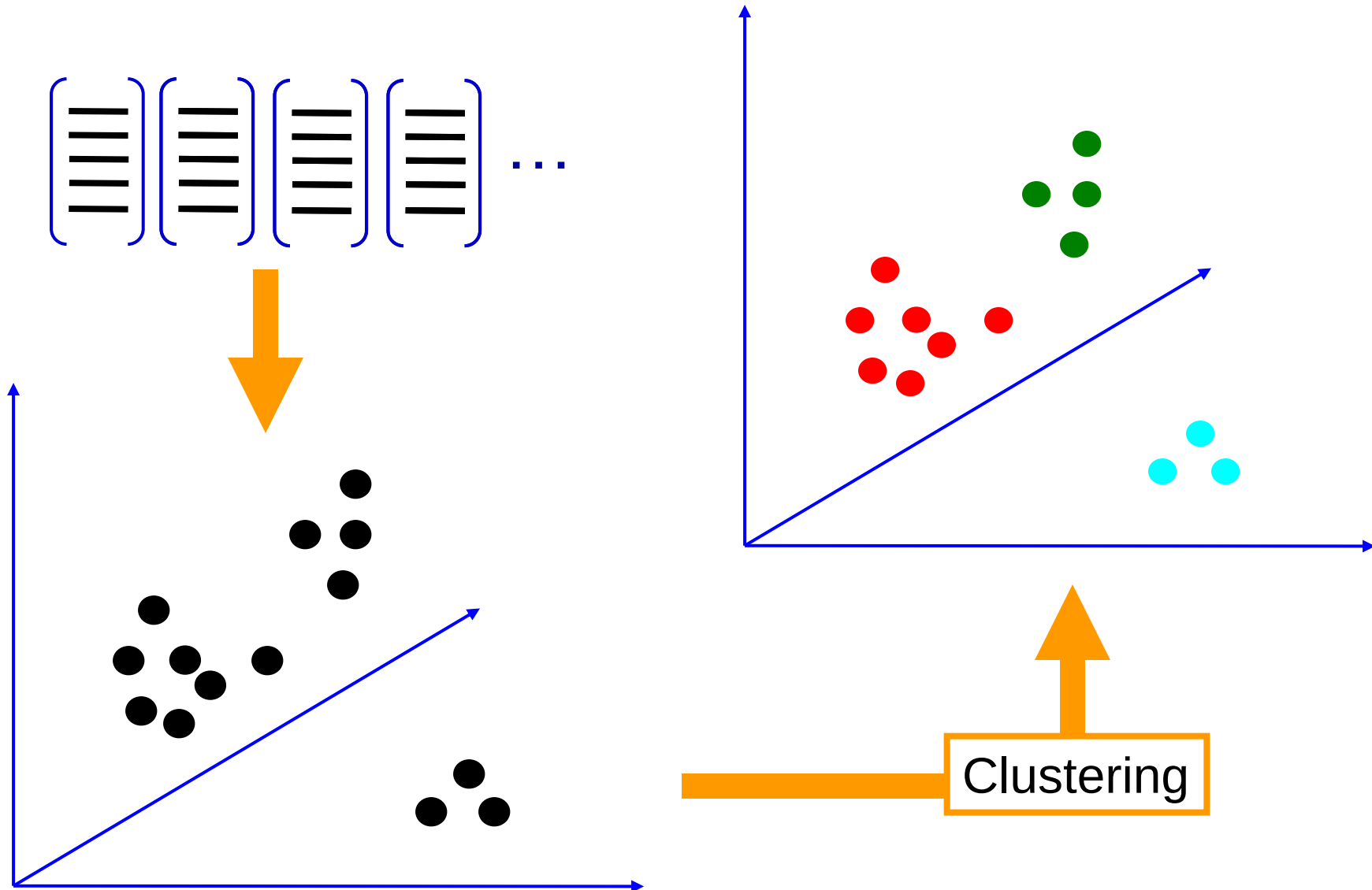


2. Learning the visual vocabulary

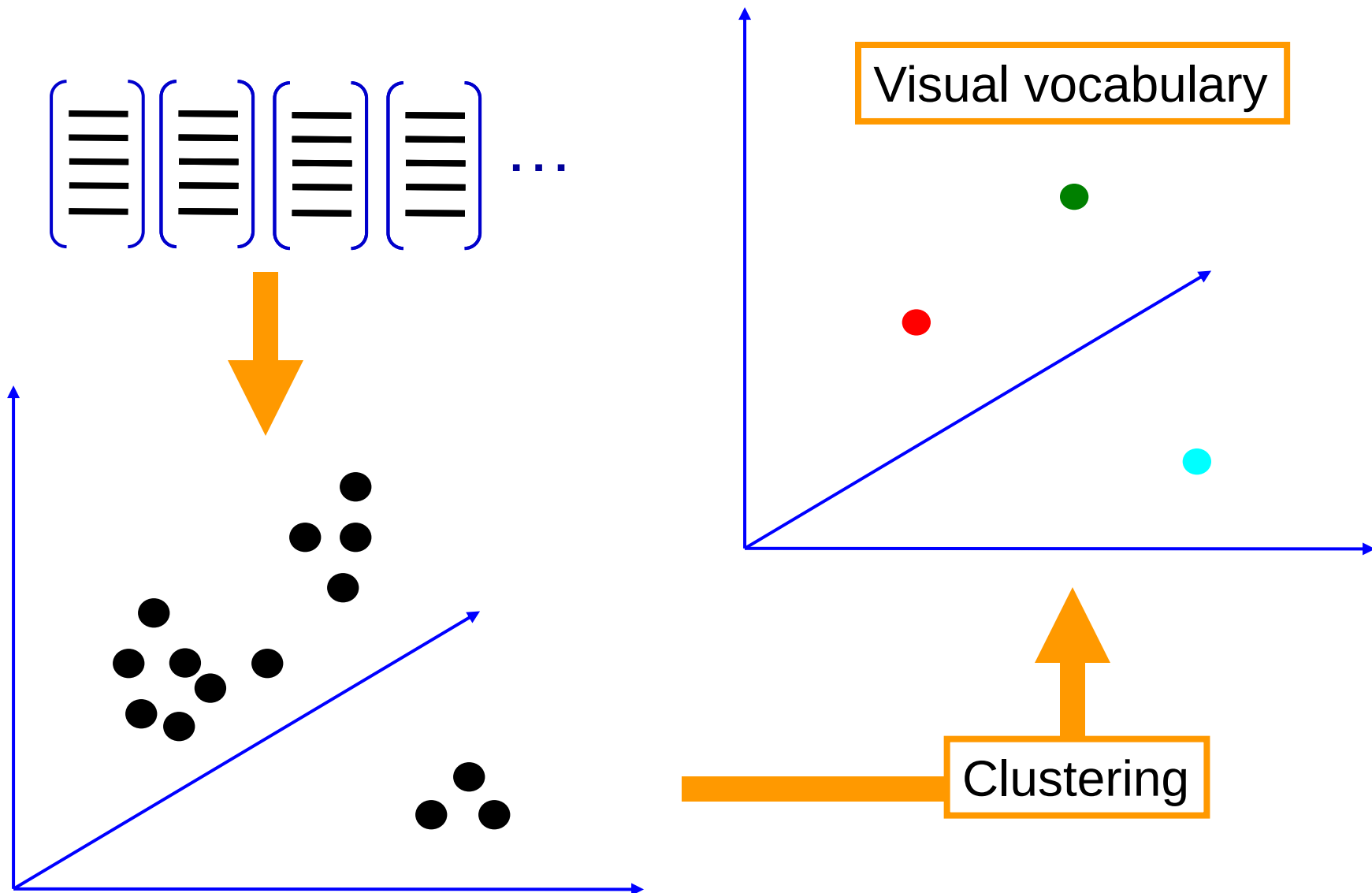


Extracted descriptors
from the training set

2. Learning the visual vocabulary



2. Learning the visual vocabulary



Recall: K-means clustering

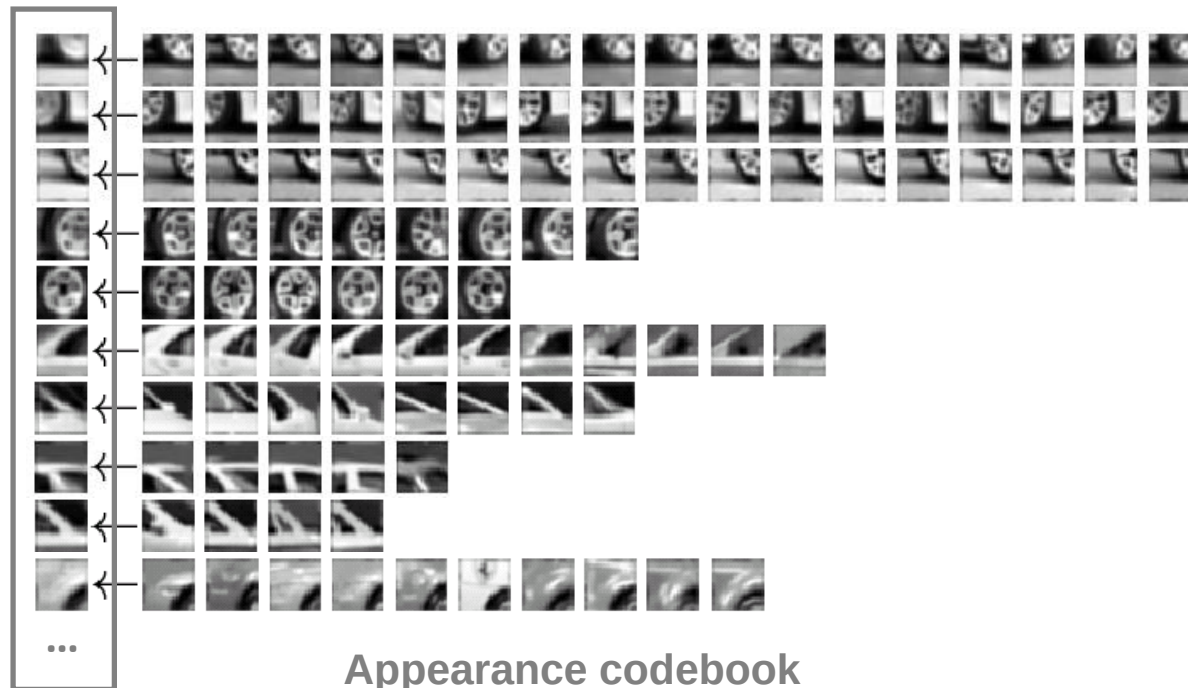
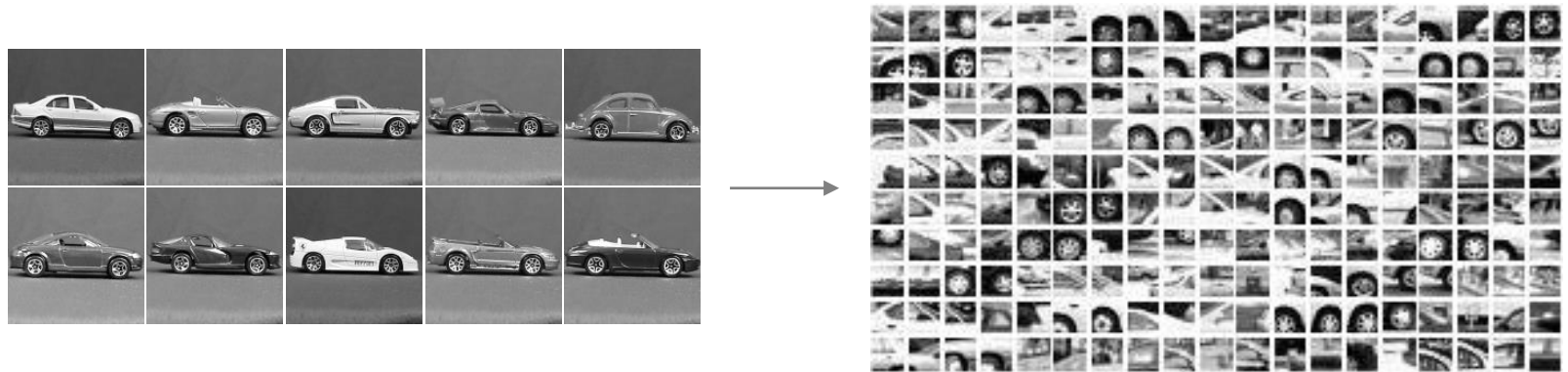
- Want to minimize sum of squared Euclidean distances between features $\mathbf{x}_i$ and their nearest cluster centers $\mathbf{m}_k$

$$D(X, M) = \sum_{\text{cluster } k} \sum_{\text{point } i \text{ in cluster } k} (\mathbf{x}_i - \mathbf{m}_k)^2$$

Algorithm:

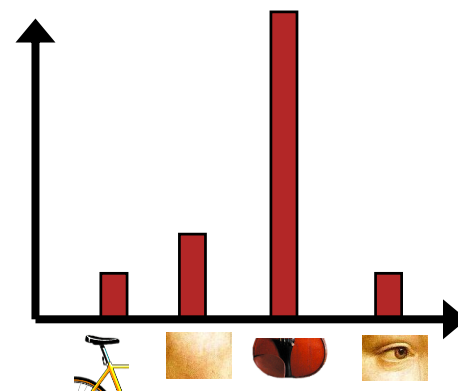
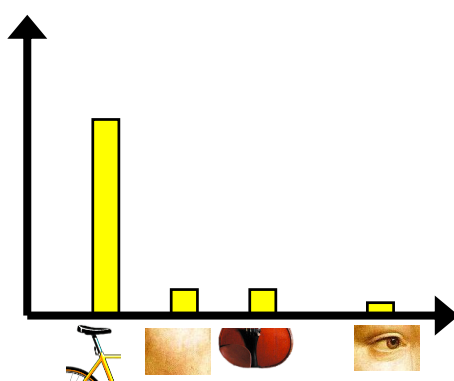
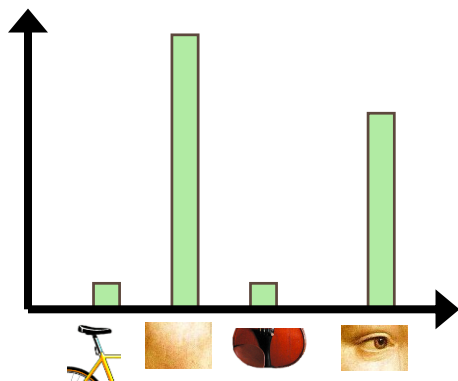
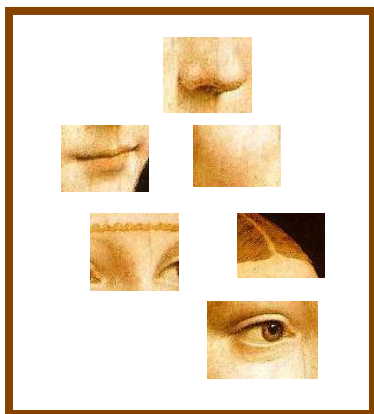
- Randomly initialize K cluster centers
- Iterate until convergence:
 - Assign each feature to the nearest center
 - Recompute each cluster center as the mean of all features assigned to it

Recall: Visual vocabularies



Bag of features: Outline

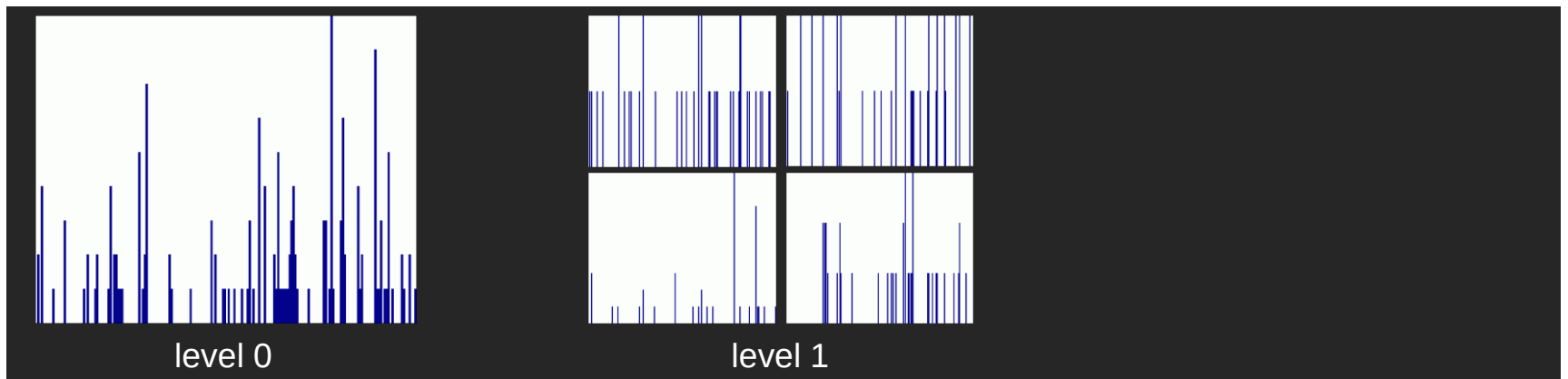
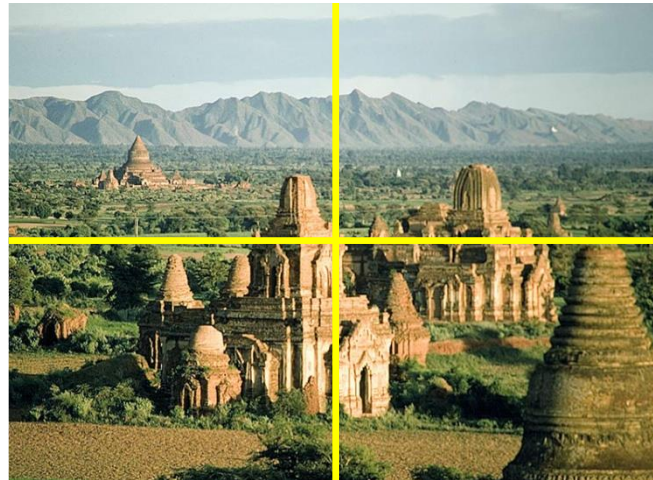
1. Extract local features
2. Learn “visual vocabulary”
3. **Quantize local features using visual vocabulary**
4. **Represent images by frequencies of “visual words”**



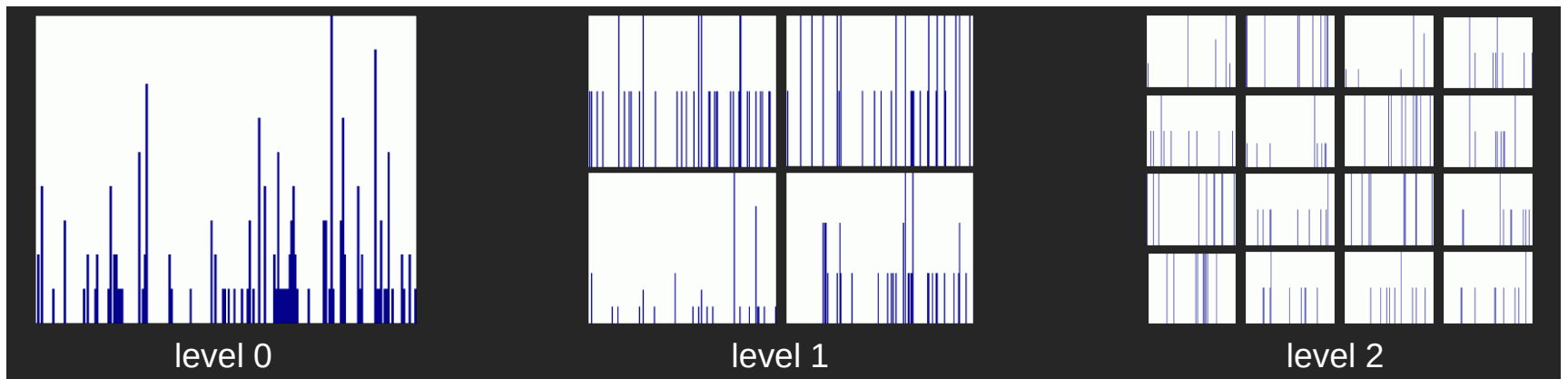
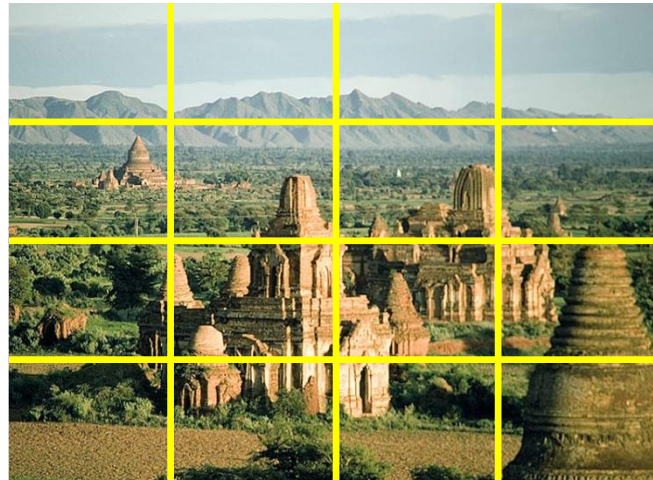
Spatial pyramids



Spatial pyramids

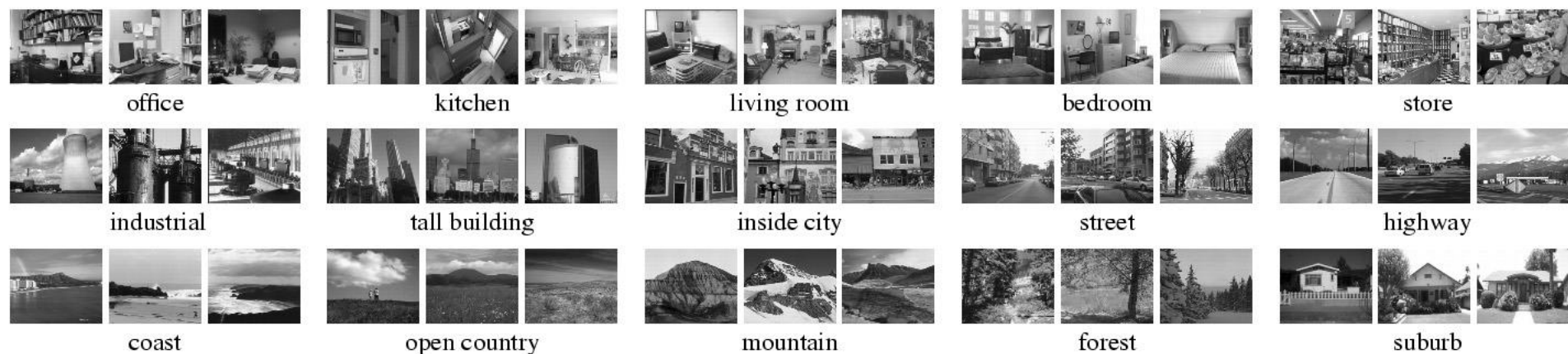


Spatial pyramids



Spatial pyramids

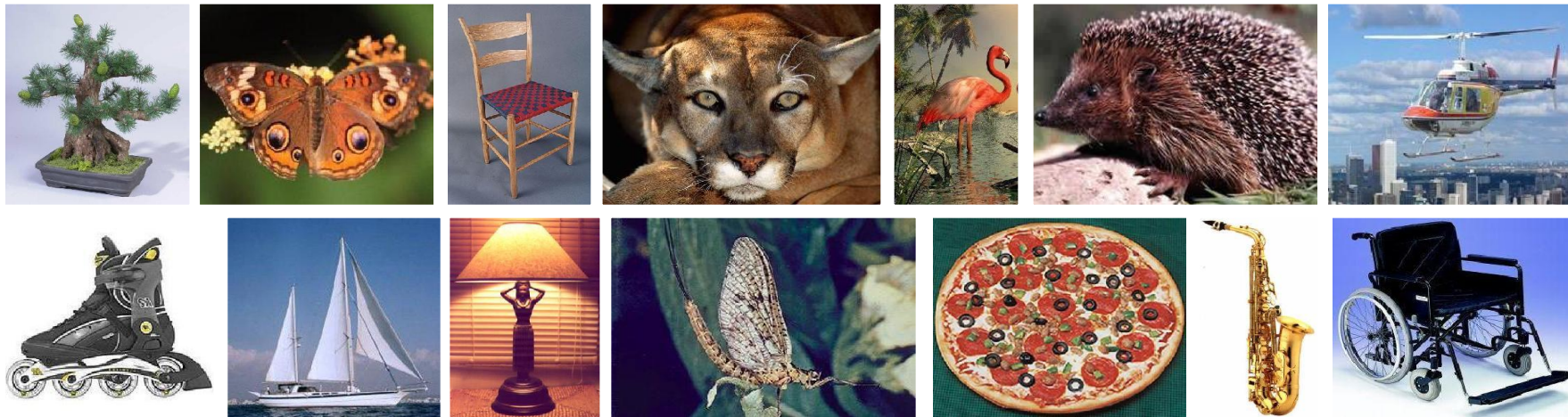
- Scene classification results



	Weak features (vocabulary size: 16)		Strong features (vocabulary size: 200)	
Level	Single-level	Pyramid	Single-level	Pyramid
0 (1×1)	45.3 $\pm$ 0.5		72.2 $\pm$ 0.6	
1 (2×2)	53.6 $\pm$ 0.3	56.2 $\pm$ 0.6	77.9 $\pm$ 0.6	79.0 $\pm$ 0.5
2 (4×4)	61.7 $\pm$ 0.6	64.7 $\pm$ 0.7	79.4 $\pm$ 0.3	81.1 $\pm$ 0.3
3 (8×8)	63.3 $\pm$ 0.8	66.8 $\pm$ 0.6	77.2 $\pm$ 0.4	80.7 $\pm$ 0.3

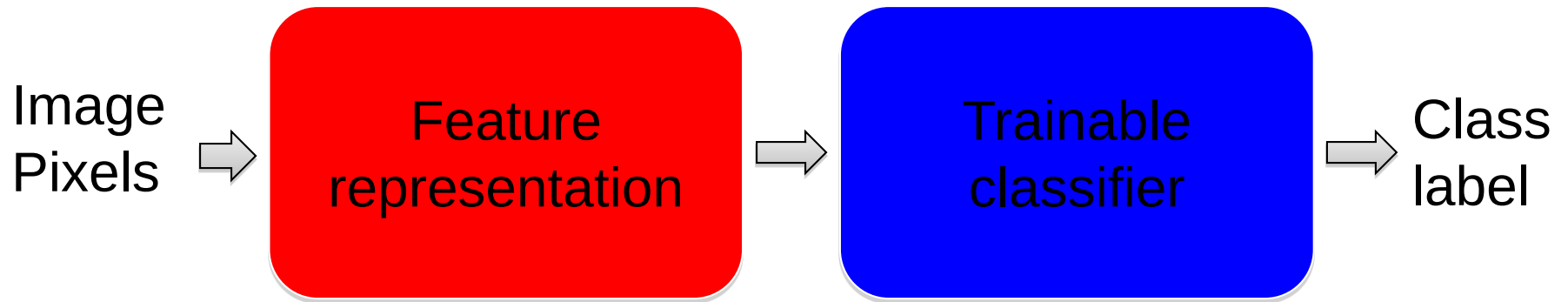
Spatial pyramids

- Caltech101 classification results



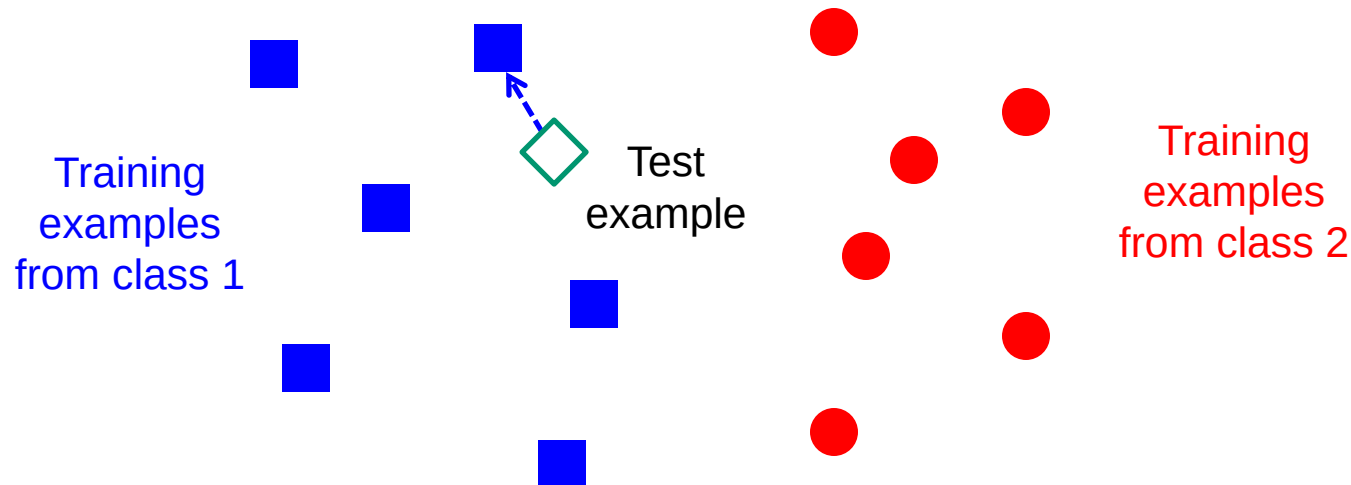
	Weak features (16)		Strong features (200)	
Level	Single-level	Pyramid	Single-level	Pyramid
0	15.5 $\pm$ 0.9		41.2 $\pm$ 1.2	
1	31.4 $\pm$ 1.2	32.8 $\pm$ 1.3	55.9 $\pm$ 0.9	57.0 $\pm$ 0.8
2	47.2 $\pm$ 1.1	49.3 $\pm$ 1.4	63.6 $\pm$ 0.9	64.6 $\pm$ 0.8
3	52.2 $\pm$ 0.8	54.0 $\pm$ 1.1	60.3 $\pm$ 0.9	64.6 $\pm$ 0.7

“Classic” recognition pipeline



- Hand-crafted feature representation
- Off-the-shelf trainable classifier

Classifiers: Nearest neighbor



$f(\mathbf{x}) = \text{label of the training example nearest to } \mathbf{x}$

All we need is a distance or similarity function for our inputs
No training required!

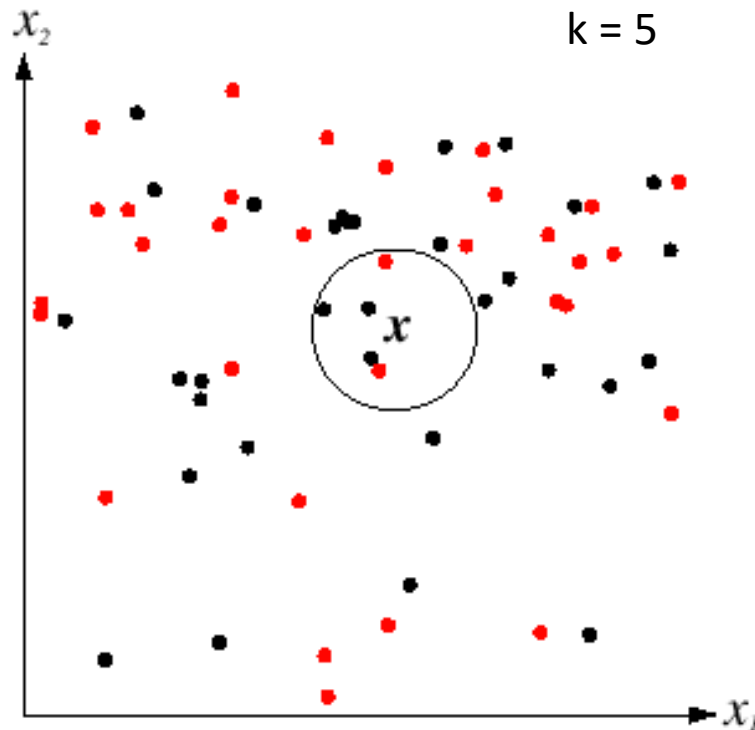
Functions for comparing histograms

- L1 distance:
$$D(h_1, h_2) = \sum_{i=1}^N |h_1(i) - h_2(i)|$$
- χ^2 distance:
$$D(h_1, h_2) = \sum_{i=1}^N \frac{(h_1(i) - h_2(i))^2}{h_1(i) + h_2(i)}$$
- Quadratic distance (*cross-bin distance*):
$$D(h_1, h_2) = \sum_{i,j} A_{ij} (h_1(i) - h_2(j))^2$$
- Histogram intersection (similarity function):

$$I(h_1, h_2) = \sum_{i=1}^N \min(h_1(i), h_2(i))$$

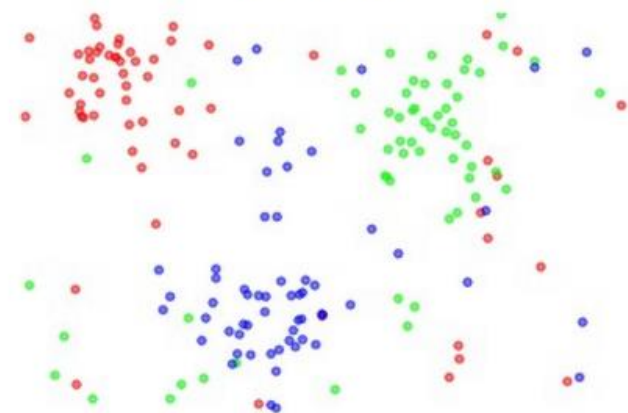
K-nearest neighbor classifier

- For a new point, find the k closest points from training data
- Vote for class label with labels of the k points

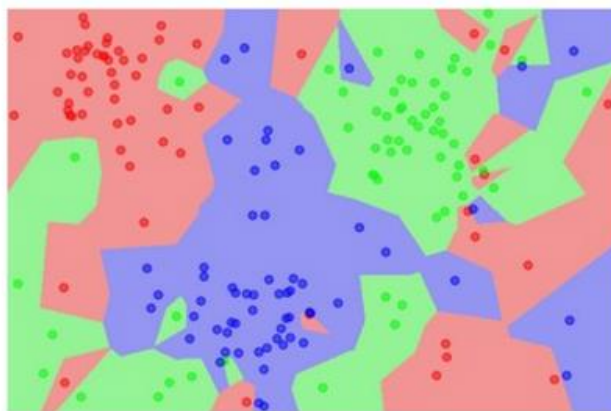


K-nearest neighbor classifier

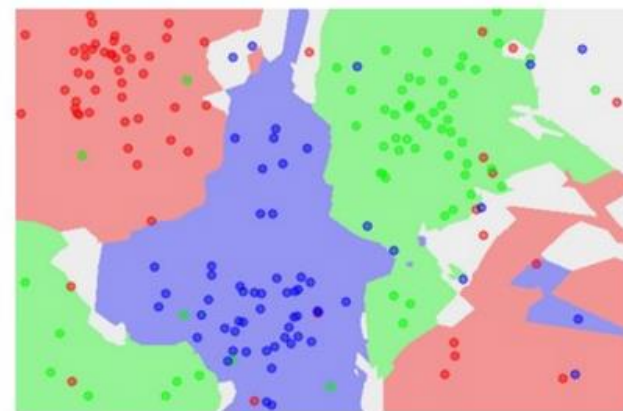
the data



NN classifier

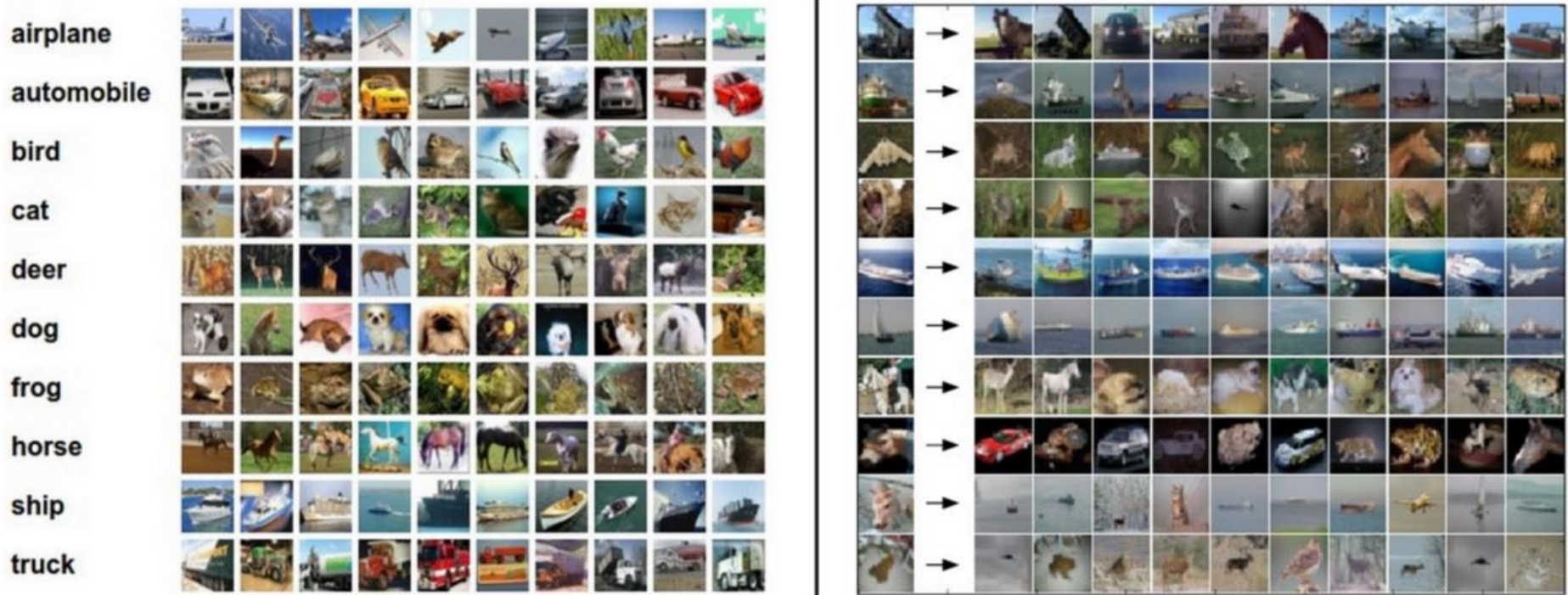


5-NN classifier



Which classifier is more robust to *outliers*?

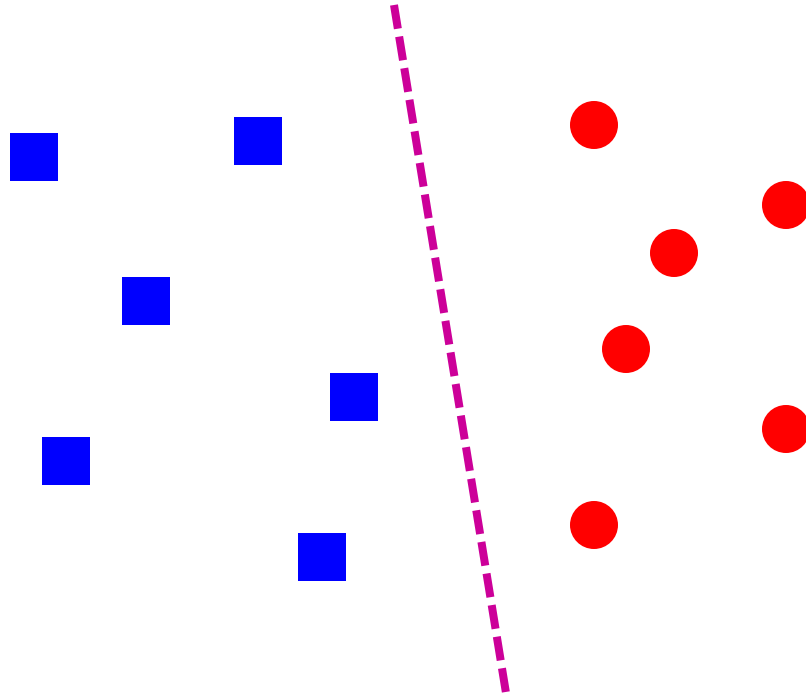
K-nearest neighbor classifier



Left: Example images from the [CIFAR-10 dataset](#). Right: first column shows a few test images and next to each we show the top 10 nearest neighbors in the training set according to pixel-wise difference.

Credit: Andrej Karpathy, <http://cs231n.github.io/classification/>

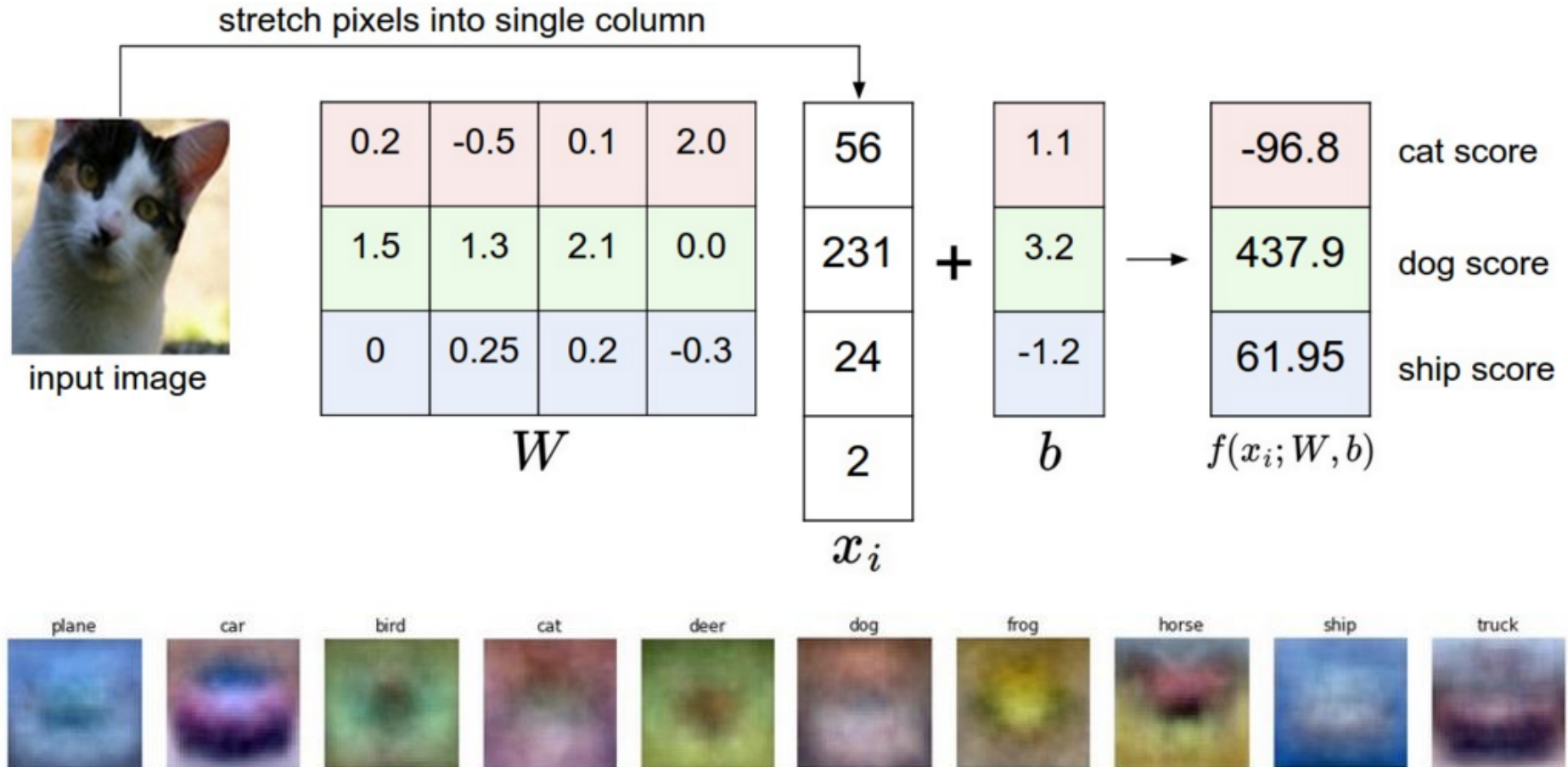
Linear classifiers



Find a *linear function* to separate the classes:

$$f(\mathbf{x}) = \text{sgn}(\mathbf{w} \cdot \mathbf{x} + b)$$

Visualizing linear classifiers



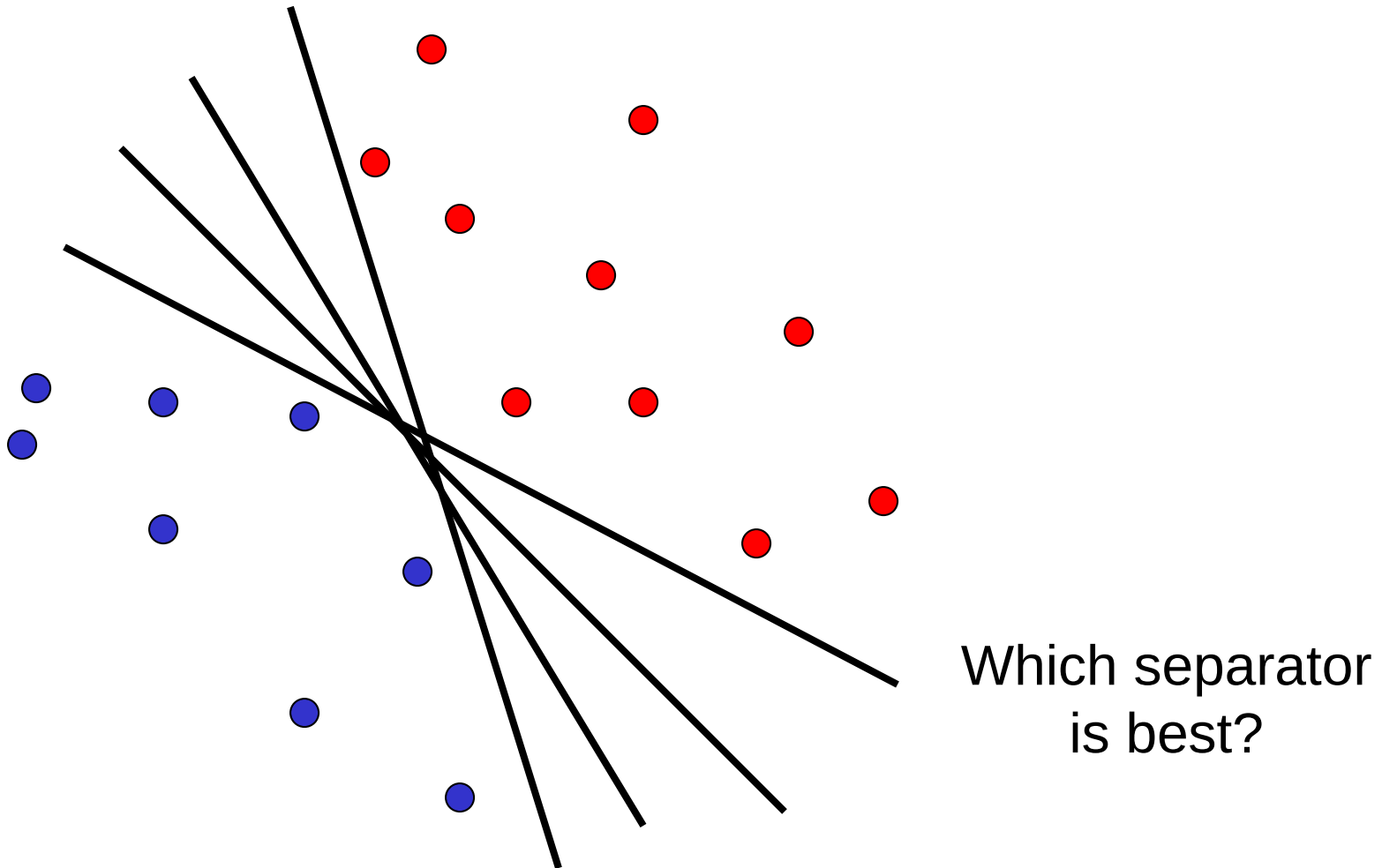
Source: Andrej Karpathy, <http://cs231n.github.io/linear-classify/>

Nearest neighbor vs. linear classifiers

- **NN pros:**
 - Simple to implement
 - Decision boundaries not necessarily linear
 - Works for any number of classes
 - *Nonparametric* method
- **NN cons:**
 - Need good distance function
 - Slow at test time
- **Linear pros:**
 - Low-dimensional *parametric* representation
 - Very fast at test time
- **Linear cons:**
 - Works for two classes
 - How to train the linear function?
 - What if data is not linearly separable?

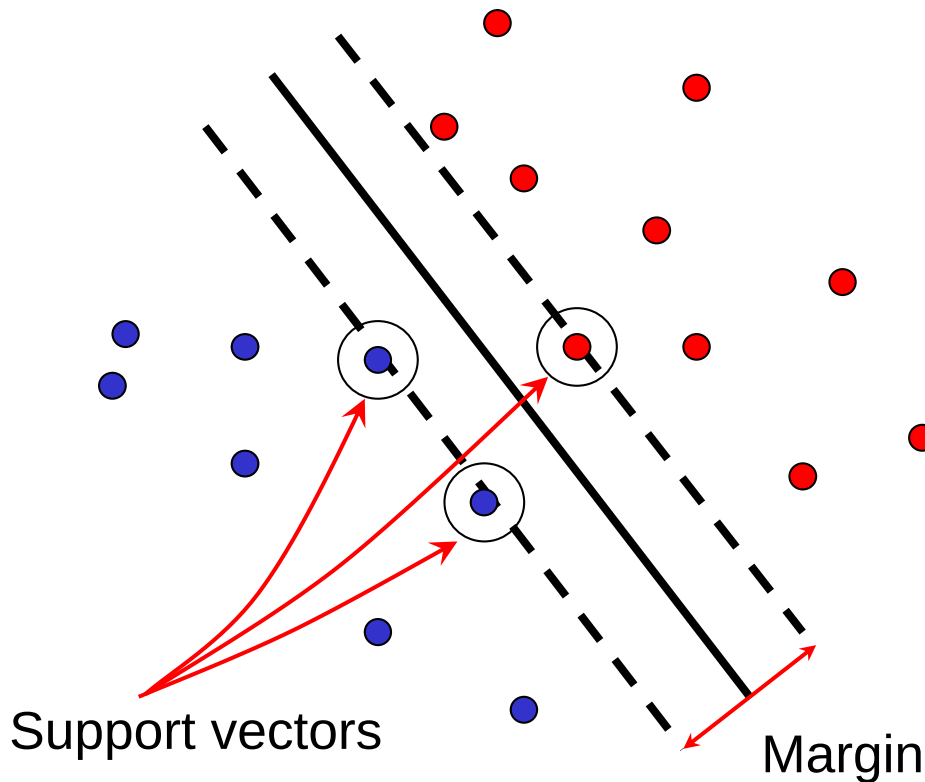
Linear classifiers

- When the data is linearly separable, there may be more than one separator (hyperplane)



Support vector machines

- Find hyperplane that maximizes the *margin* between the positive and negative examples



$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

$$\text{For support vectors, } \mathbf{x}_i \cdot \mathbf{w} + b = \pm 1$$

$$\text{Distance between point and hyperplane: } \frac{|\mathbf{x}_i \cdot \mathbf{w} + b|}{\|\mathbf{w}\|}$$

$$\text{Therefore, the margin is } 2 / \|\mathbf{w}\|$$

Finding the maximum margin hyperplane

1. Maximize margin $2 / \|\mathbf{w}\|$
2. Correctly classify all training data:

$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

Quadratic optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$$

SVM parameter learning

- Separable data: $\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$

Maximize
margin

Classify training data correctly

- Non-separable data:

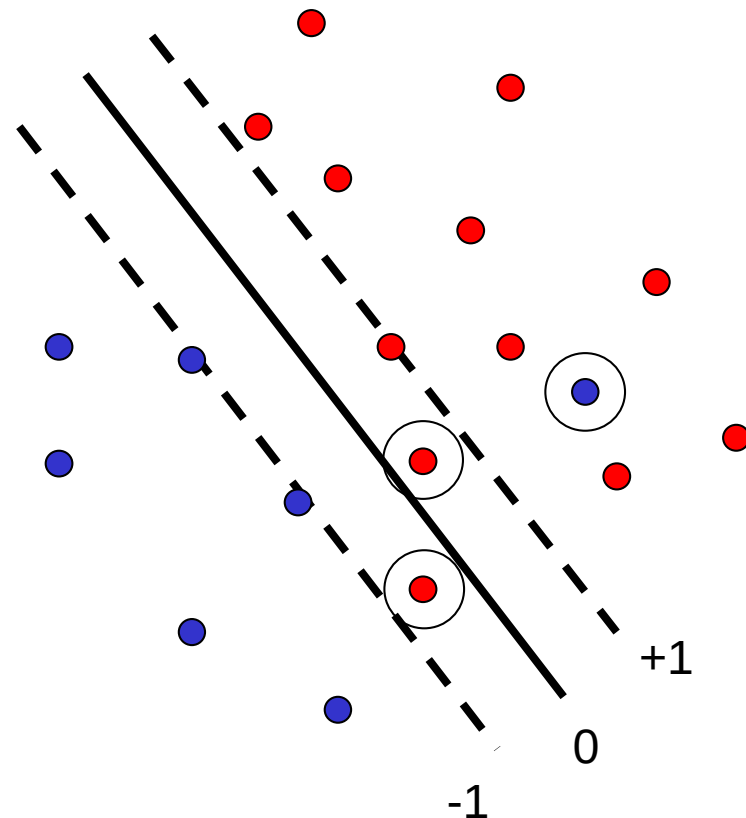
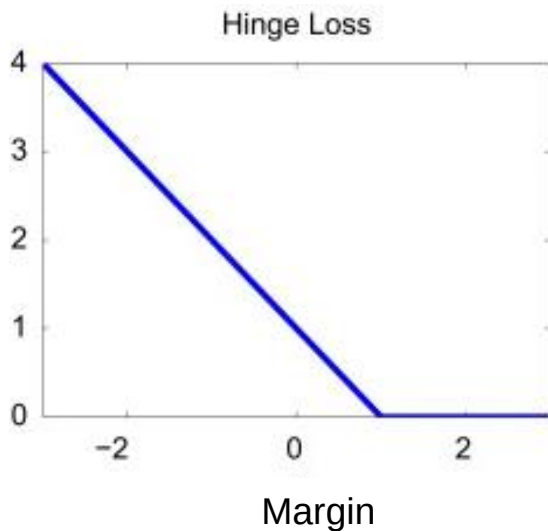
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))$$

Maximize
margin

Minimize classification mistakes

SVM parameter learning

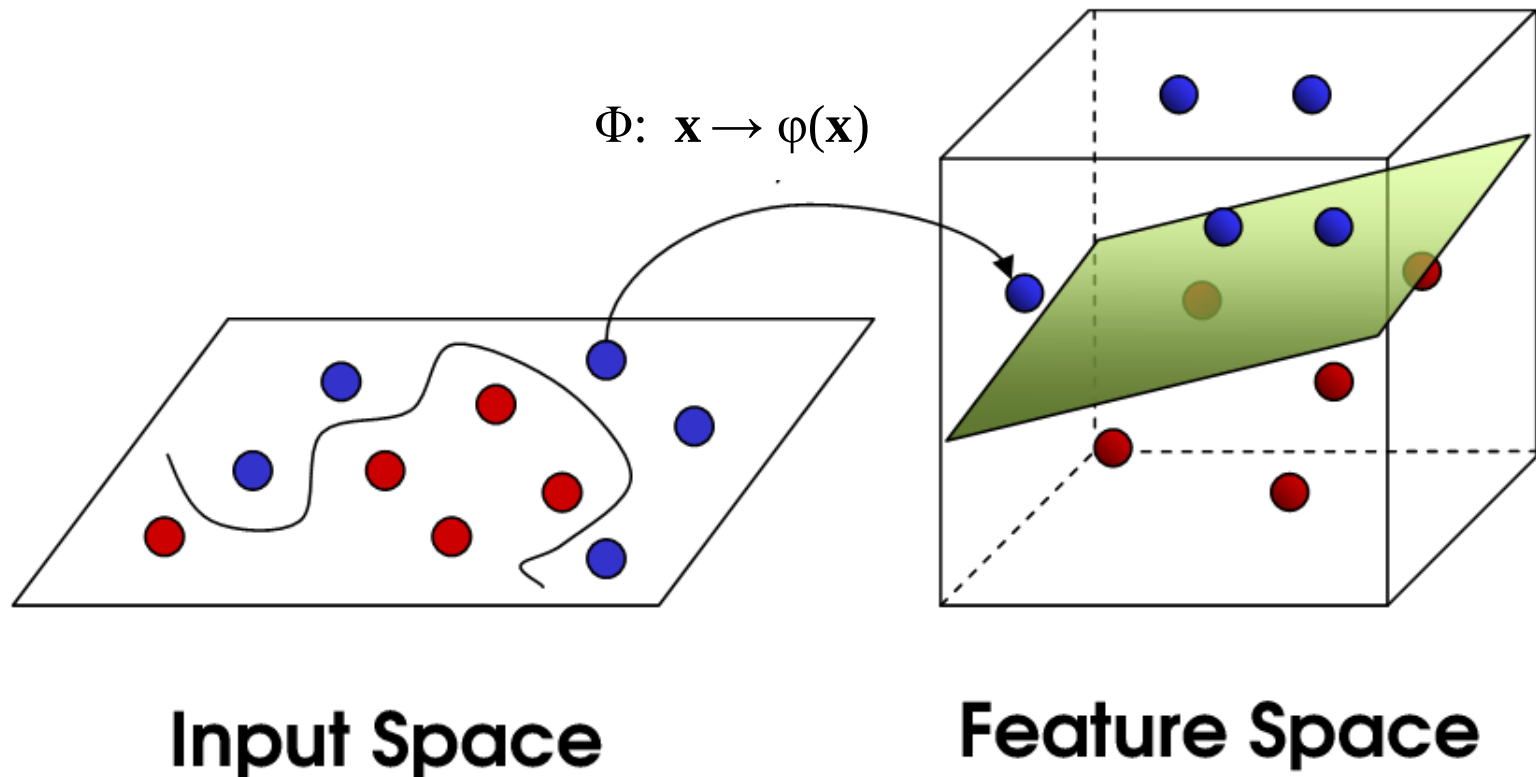
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))$$



Demo: <http://cs.stanford.edu/people/karpathy/svmjs/demo>

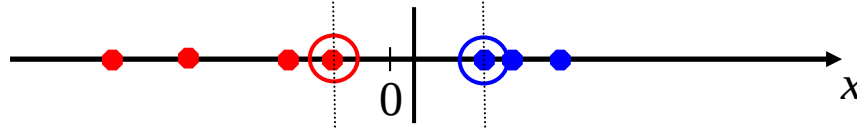
Nonlinear SVMs

- **General idea:** the original input space can always be mapped to some higher-dimensional feature space where the training set is separable

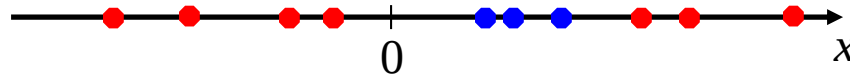


Nonlinear SVMs

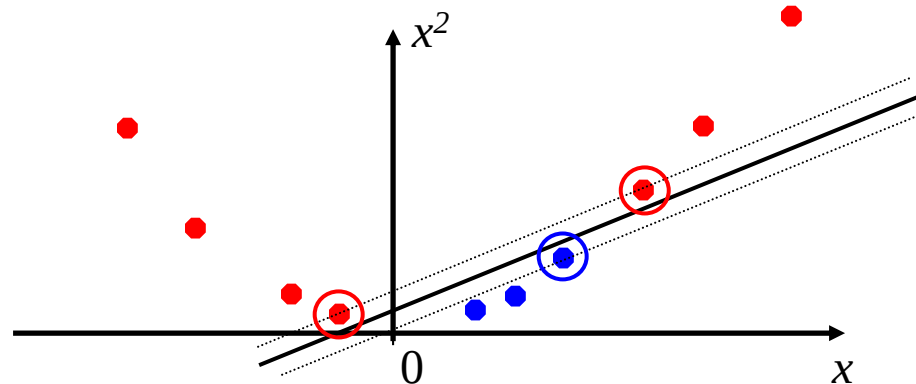
- Linearly separable dataset in 1D:



- Non-separable dataset in 1D:



- We can map the data to a *higher-dimensional space*:



The kernel trick

- **General idea:** the original input space can always be mapped to some higher-dimensional feature space where the training set is separable
- **The kernel trick:** instead of explicitly computing the lifting transformation $\varphi(\mathbf{x})$, define a kernel function K such that

$$K(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{y})$$

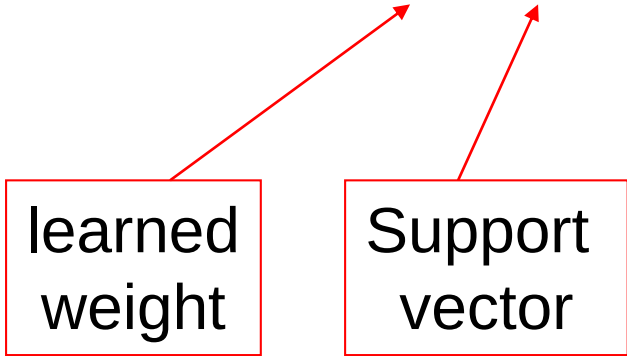
(to be valid, the kernel function must satisfy *Mercer's condition*)

The kernel trick

- Linear SVM decision function:

$$\mathbf{w} \cdot \mathbf{x} + b = \sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b$$

learned
weight



Support
vector

The kernel trick

- Linear SVM decision function:

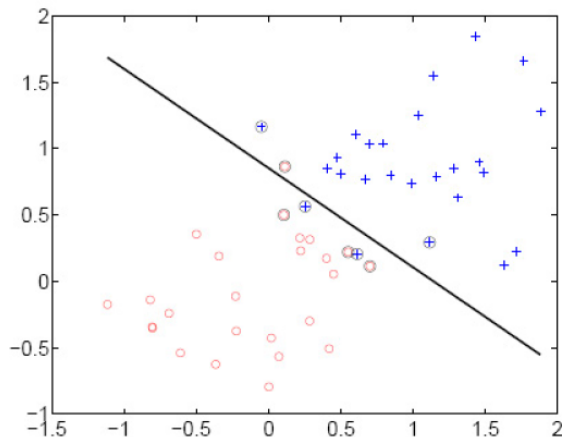
$$\mathbf{w} \cdot \mathbf{x} + b = \sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b$$

- Kernel SVM decision function:

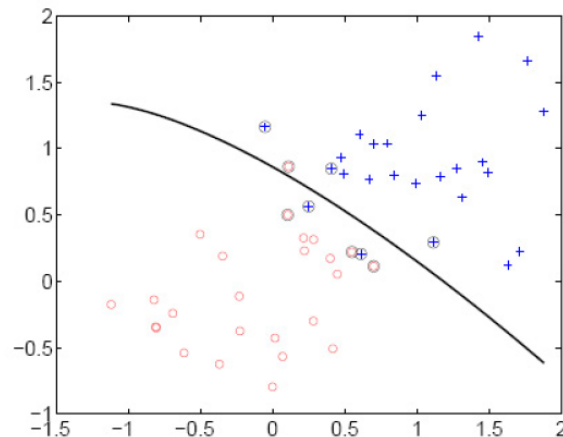
$$\sum_i \alpha_i y_i \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}) + b = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

- This gives a nonlinear decision boundary in the original feature space

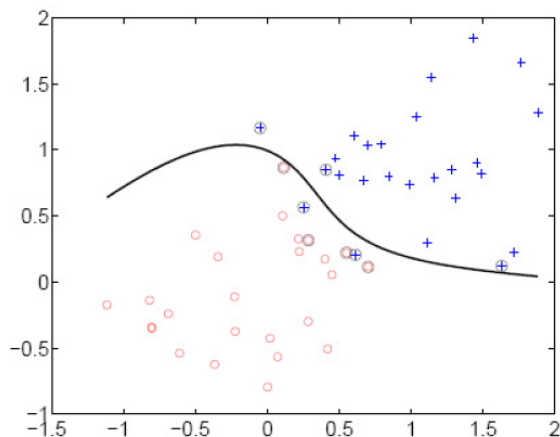
Polynomial kernel: $K(\mathbf{x}, \mathbf{y}) = (c + \mathbf{x} \cdot \mathbf{y})^d$



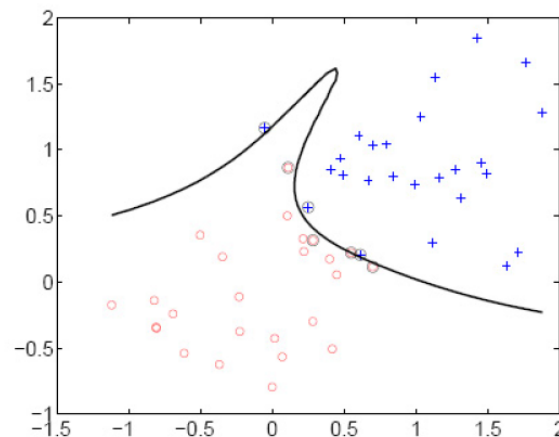
linear



2^{nd} order polynomial



4^{th} order polynomial

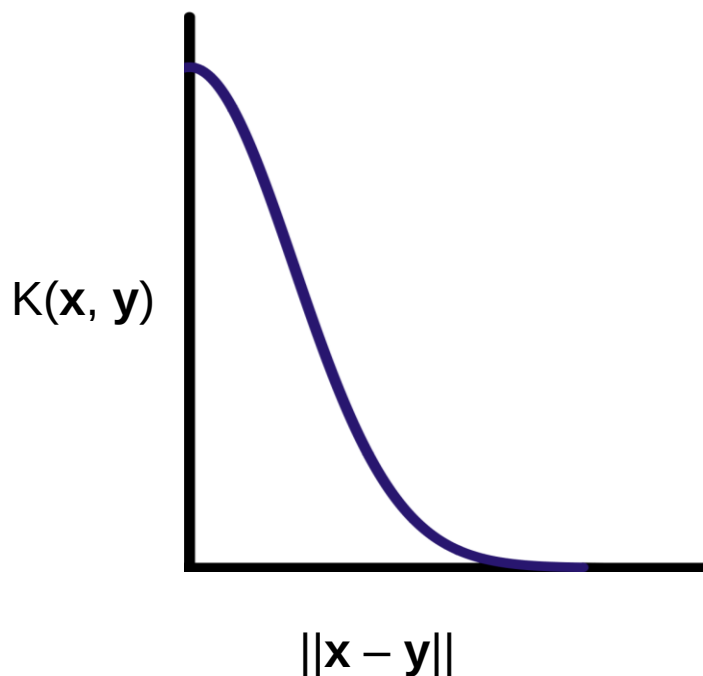


8^{th} order polynomial

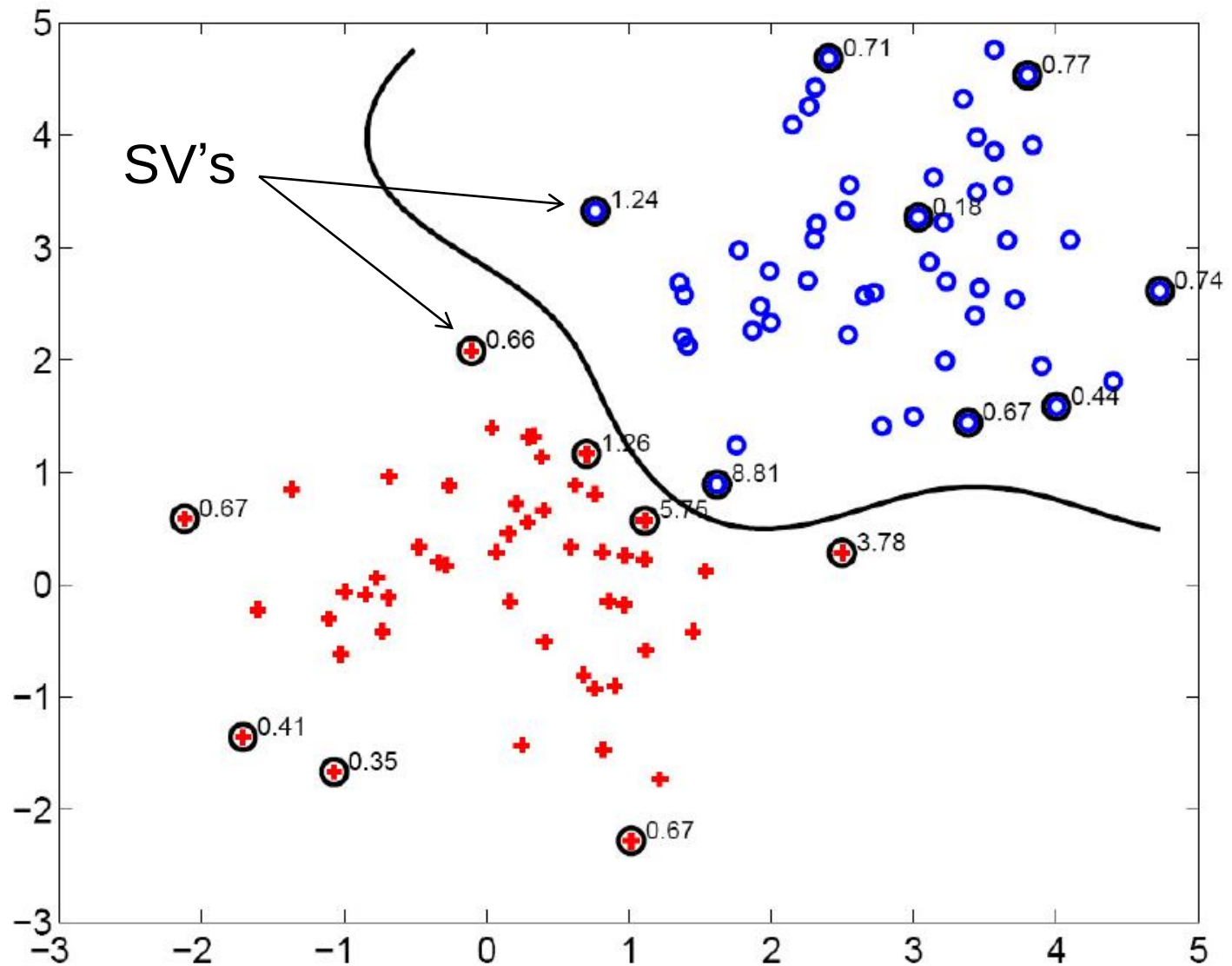
Gaussian kernel

- Also known as the radial basis function (RBF) kernel:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{1}{\sigma^2} \|\mathbf{x} - \mathbf{y}\|^2\right)$$



Gaussian kernel



Kernels for bags of features

- Histogram intersection:

$$K(h_1, h_2) = \sum_{i=1}^N \min(h_1(i), h_2(i))$$

- Square root (Bhattacharyya kernel):

$$K(h_1, h_2) = \sum_{i=1}^N \sqrt{h_1(i)h_2(i)}$$

- Generalized Gaussian kernel:

$$K(h_1, h_2) = \exp\left(-\frac{1}{A} D(h_1, h_2)^2\right)$$

- D can be L1 distance, Euclidean distance, χ^2 distance, etc.

SVMs: Pros and cons

- Pros

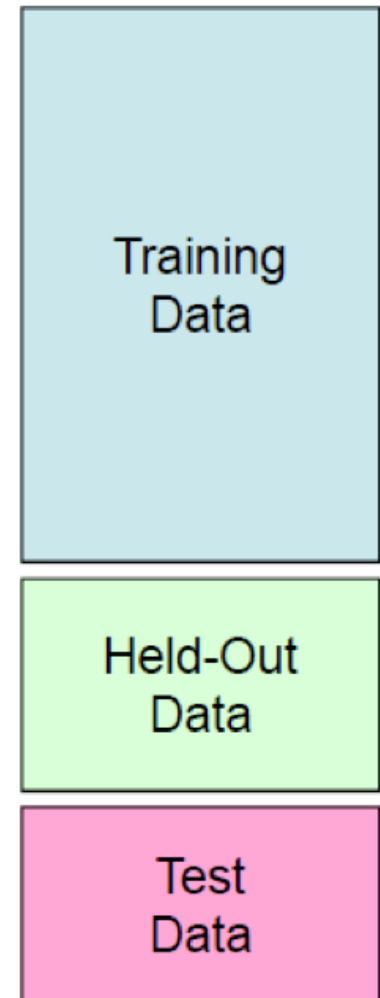
- Kernel-based framework is very powerful, flexible
- Training is convex optimization, globally optimal solution can be found
- Amenable to theoretical analysis
- SVMs work very well in practice, even with very small training sample sizes

- Cons

- No “direct” multi-class SVM, must combine two-class SVMs (e.g., with one-vs-others)
- Computation, memory (esp. for nonlinear SVMs)

Best practices for training classifiers

- Goal: obtain a classifier with **good generalization** or performance on never before seen data
1. Learn *parameters* on the **training set**
 2. Tune *hyperparameters* (implementation choices) on the *held out validation set*
 3. Evaluate performance on the **test set**
 - Crucial: do not peek at the test set when iterating steps 1 and 2!



What's the big deal?

Baidu admits cheating in international supercomputer competition



Baidu recently apologised for violating the rules of an international supercomputer test in May, when the Chinese search engine giant claimed to beat both Google and Microsoft on the ImageNet image-recognition test.

 By [Cyrus Lee](#) | June 10, 2015 -- 00:15 GMT (17:15 PDT) | Topic: [China](#)

TECHNOLOGY

The New York Times

Computer Scientists Are Astir After Baidu Team Is Barred From A.I. Competition

By [JOHN MARKOFF](#) JUNE 3, 2015



Baidu caught gaming recent supercomputer performance test

 by [Andrew Tarantola](#) | [@terrortola](#) | June 3rd 2015 At 11:09pm



IMAGENET Large Scale Visual Recognition Challenge (ILSVRC)

Date: June 2, 2015

Dear ILSVRC community,

This is a follow up to the announcement on [May 19, 2015](#) with some more details and the status of the test server.

During the period of November 28th, 2014 to May 13th, 2015, there were at least 30 accounts used by a team from Baidu to submit to the test server at least 200 times, far exceeding the specified limit of two submissions per week. This includes short periods of very high usage, for example with more than 40 submissions over 5 days from March 15th, 2015 to March 19th, 2015. Figure A below shows submissions from ImageNet accounts known to be associated with the team in question. Figure B shows a comparison to the activity from all other accounts.

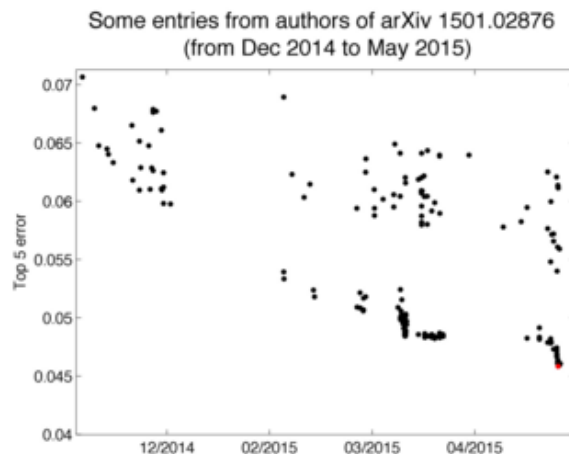


Figure A

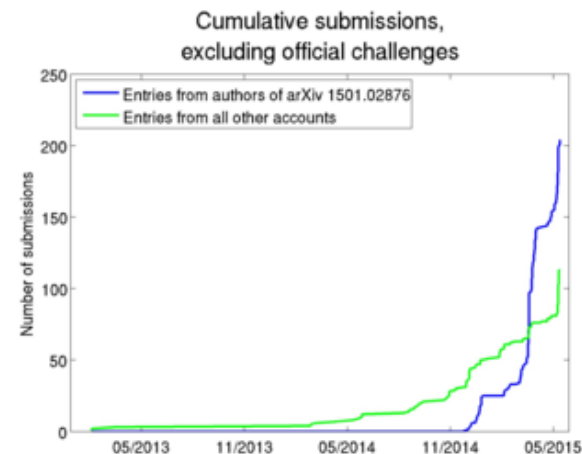


Figure B

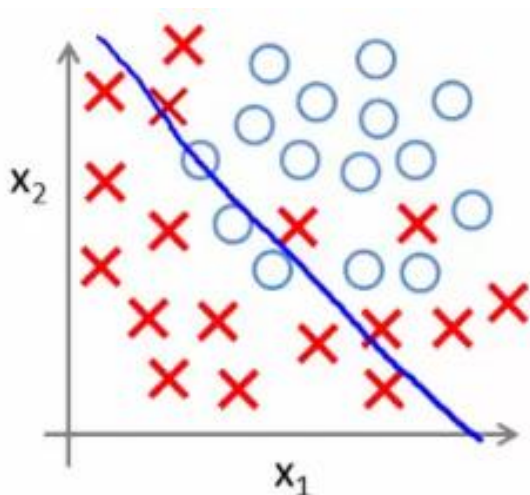
The results obtained during this period are reported in a [recent arXiv paper](#). Because of the violation of the regulations of the test server, these results may not be directly comparable to results obtained and reported by other teams. To make this clear, by exploiting the ability to test many slightly different solutions on the test server it is possible to 1) select the best out of a set of very similar solutions based on test performance and achieve a small but potentially significant advantage and 2) choose methods for further research and development based directly on the test data instead of using only the training and validation data for such choices.

<http://www.image-net.org/challenges/LSVRC/announcement-June-2-2015>

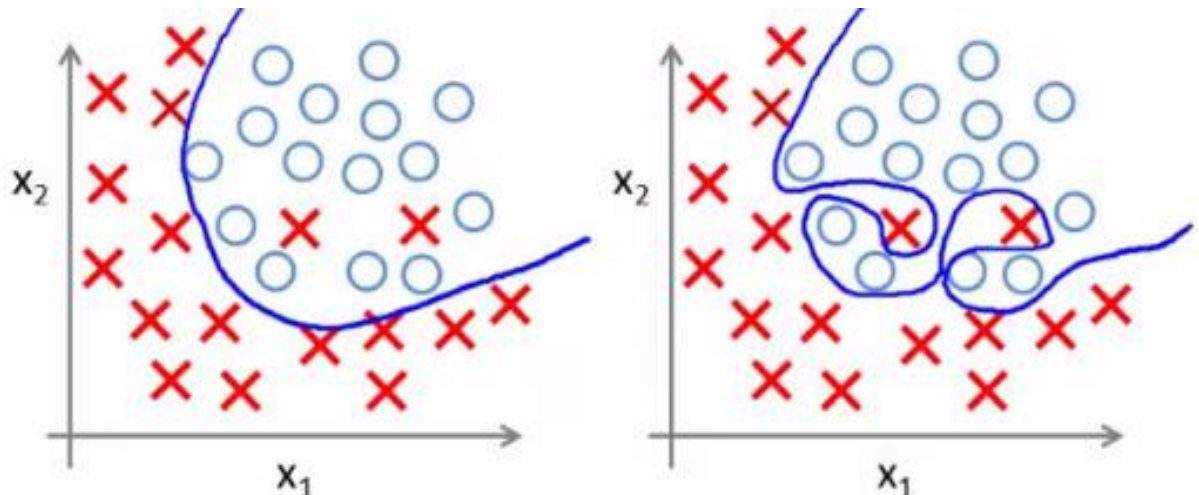
Bias-variance tradeoff

- Prediction error of learning algorithms has two main components:
 - **Bias:** error due to simplifying model assumptions
 - **Variance:** error due to randomness of training set
- **Bias-variance tradeoff** can be controlled by turning “knobs” that determine model complexity

High bias, low variance



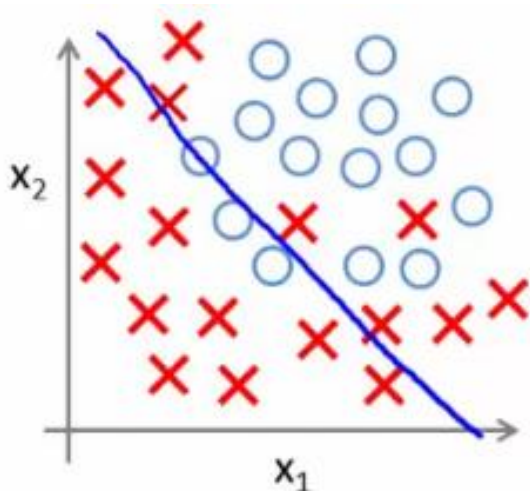
Low bias, high variance



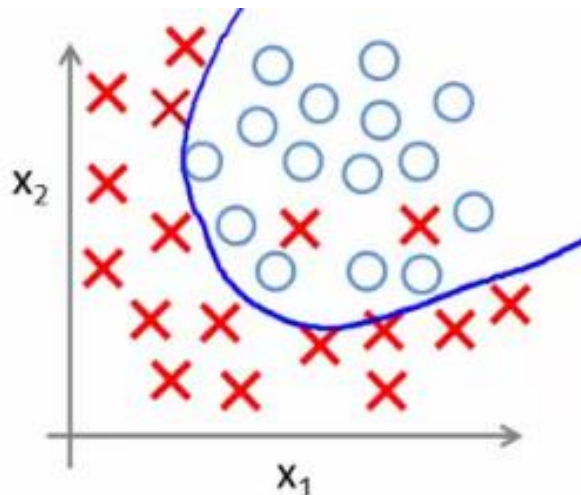
Underfitting and overfitting

- **Underfitting:** training and test error are both *high*
 - Model does an equally poor job on the training and the test set
 - The model is too “simple” to represent the data or the model is not trained well
- **Overfitting:** Training error is *low* but test error is *high*
 - Model fits irrelevant characteristics (noise) in the training data
 - Model is too complex or amount of training data is insufficient

Underfitting



Good tradeoff



Overfitting

